Zona Python Bloque 01

Temario aula

Table of contents

1. Zona Python 3
1.1 Referencias 3
2. Bl. 01 4
2.1 Introducción a Python 4
2.2 Estructuras control 13
2.3 Estructuras Datos 28
2.4 Ejercicios I 46
2.5 Módulos/funciones 54
2.6 Introducción a Python - Excepciones 60
2.7 Ejercicios II 65
2.8 Prácticas 69
3. Tools 74
3.1 Introducción a Python - Linters 74

1. Zona Python

Apuntes y complementos Módulo sobre Python

1.1 Referencias

- Tutorial Python de W3Schools
- Curso online en Netacad: Fundamentos de Python 1

2. Bl. 01

2.1 Introducción a Python

En estos ejemplos se presentan algunos de los elementos básicos que utilizaremos continuamente al programar en Python: **variables**, **tipos de datos**, **cadenas de texto**, **valores booleanos** y **conversión de tipos**.

2.1.1 1. Entorno de trabajo

- Sistema operativo Linux con Python ya instalado
- Editor de texto Visual Studio Code
- Trabajo en terminal

2.1.2 2. Variables y tipos de datos

Una **variable** permite guardar un valor para utilizarlo posteriormente. En Python no es necesario indicar de antemano el tipo de una variable: este se determina a partir del valor que le asignamos.

2.1. Código completo

```
# Variables, asignación (=) y tipos (int, float, str...)

# Asignación
saldo_total = 254.55  # float
email = 'ana@local.com'  # str

print("saldo_total") # imprime cadena
print(saldo_total)   # imprime valor de la variable
  print(email)   # error indentación: el programm se detiene

# Imprime tipo de las variables usando type(variable)
print("tipo de email: ", type(email)  )
print("tipo de saldo_total: ", type(saldo_total)  )
# print puede recibir varias expresiones separadas por comas

# Variables pueden cambiar de tipo
#  NO es algo recomdable
email = 4  # pasa de string a integer
print( "Tipo email: ", type(email)  )
print("El email es ", email, " y el saldo es ", saldo_total, "\n")
```

2.2. Asignar valores a variables

Las primeras instrucciones crean dos variables:

```
saldo_total = 254.55
email = 'ana@local.com'
```

`saldo_total` contiene un número decimal, por lo que su tipo es `float`. `email` contiene texto, por lo que su tipo es `str`.

El símbolo `=` es el **operador de asignación**: guarda el valor situado a la derecha en la variable situada a la izquierda.

> **Asignación y comparación no son lo mismo**
>
> En Python, `=` se utiliza para **asignar** un valor. Más adelante veremos que `==` se utiliza para **comparar** dos valores.

2.3. Imprimir texto o imprimir una variable

Estas dos instrucciones parecen muy similares, pero no hacen lo mismo:

```
print("saldo_total")
print(saldo_total)
```

La primera imprime literalmente el texto:

```
saldo_total
```

La segunda consulta el contenido de la variable e imprime:

```
254.55
```

Las comillas son, por tanto, importantes: `"saldo_total"` es una cadena de texto, mientras que `saldo_total` es el nombre de una variable.

2.4. Un error de indentación

El ejemplo contiene intencionadamente esta línea:

```
  print(email)
```

Hay espacios al comienzo de la línea sin que exista ningún bloque que justifique esa indentación. Python considera la indentación parte de la sintaxis y detendrá el programa con un `IndentationError`.

Mientras ese error exista, las instrucciones posteriores **no llegarán a ejecutarse**.

La línea correcta sería:

```
print(email)
```

> **La indentación importa**
>
> En Python no utilizamos la indentación solo para que el programa quede más bonito. Sirve para indicar qué instrucciones pertenecen a determinados bloques de código.

2.5. Consultar el tipo de un dato

La función `type()` permite saber de qué tipo es un valor o una variable:

```
print("tipo de email: ", type(email))
print("tipo de saldo_total: ", type(saldo_total))
```

Una vez corregido el error anterior, veremos que inicialmente:

- `email` es de tipo `str`.
- `saldo_total` es de tipo `float`.

Además, `print()` puede recibir varios valores separados por comas:

```
print("El email es ", email, " y el saldo es ", saldo_total)
```

2.6. Una variable puede cambiar de tipo

Python permite hacer esto:

```
email = 4
```

Aunque antes `email` contenía una cadena, ahora contiene un entero (`int`).

Es posible hacerlo, pero normalmente **no es una buena práctica** reutilizar una variable para almacenar información de naturaleza completamente diferente. El nombre `email`, además, deja de describir correctamente lo que contiene.

2.7. Posibles mejoras

- Corregir la indentación de `print(email)`.
- Evitar que una variable llamada `email` termine almacenando el número `4`.
- Utilizar nombres de variables que describan claramente su contenido.
- Mantener un estilo uniforme en los espacios y llamadas a `print()`.

2.1.3 3. Trabajo con cadenas de texto

Las cadenas (`str`) representan texto. Python proporciona muchas operaciones para mostrarlas, transformarlas, buscar contenido en ellas y recorrer sus caracteres.

3.1. Código completo

```
# Ejemplo uso format en print
nombre = "Alfredo Zumberg"
edad = 23
altura = 1.915
print(nombre, "su edad es", edad, "y altura", altura)
# Uso de format con cadenas
print(f"{nombre} su edad es {edad} y altura {altura:.1f}")


# Algunos métodos de string

saludo = "Hola Mundo"
print("Valor de saludo:", saludo)

print("Resultado de saludo.lower():", saludo.lower())
print("Valor de saludo:", saludo)


# No modifica cadena original.
# Si queremos modificar hace falta asignación
saludo = saludo.lower()
print("Valor de saludo:", saludo)

# Replace, lo mismo..
print('Resultado de saludo.replace("mundo", "universo"):' , saludo.replace("mundo", "universo"))
print("Valor de saludo:", saludo)

# el tipo str se puede tratar como array o lista:
saludo ="¡Hola mundo!"
print("Primer carácter:", saludo[0])
print("Segundo carácter:",saludo[1])
print("Longitud saludo:", len(saludo))

# Buscar en un str: in y not in
texto = "The best thiNgs  life are free!"
busca = "FREE"

if busca.lower() in texto.lower():
    print(f"{busca} sí está en {texto}")
else:
    print(f"{busca} NO está en {texto}")

# o en negativo y sin ser case sensitve
if busca not in texto:
    print(f"{busca} NO está en {texto}")
else:
    print(f"{busca} sí está en {texto}")


# Concatenar str
nombre = "Raquel"
apellidos = "García Ross"
print(nombre + " " + apellidos)  # el operador + con cadenas concatena

nombreCompleto = apellidos + ", " + nombre
print(nombreCompleto)

# Extraer parte del string: slice

# La sintaxis del slicing en Python es string[inicio:fin:paso]:
#   - inicio: Por defecto empieza desde el extremo del string.
#   - fin: Por defecto recorre hasta el otro extremo.
#   - paso: por defecto 1.

saludo = "Hola Mundo"
print(saludo[:4])  # "Hola"
print(saludo[5:])  # "Mundo"
print(saludo[::2]) # de dos en dos => "Hl ud"
```

```
# Para invertir una cadena se usa salto negativo
print(saludo[::-1]) # en order inverso => "odnuM aloH"



# Se muestra ahora el uso de cadenas con bucle for

nombre = "www.aulas.local"
# a) Recorrer cada caracter de un str
for x in nombre:
    print(x)

# b) Recorrer cada caracter de un str
# usando el índice o posición
# se introduce el uso de range()
for i in range(len(nombre)):
    print(f"{i}: {nombre[i]}")


# Ejemplo de ejecución de range
# >>> for i in range(5):
# ...     print(i)
# ...
# 0
# 1
# 2
# 3
# 4
# >>>
```

3.2. Mostrar valores con `print()` y f-strings

Podemos imprimir varios valores separándolos mediante comas:

```
print(nombre, "su edad es", edad, "y altura", altura)
```

Otra posibilidad muy cómoda son las **f-strings**:

```
print(f"{nombre} su edad es {edad} y altura {altura:.1f}")
```

Todo lo que aparece entre `{}` se sustituye por su valor.

En el caso de:

```
{altura:.1f}
```

se solicita mostrar el número decimal con **una cifra después del punto decimal**.

3.3. Métodos de las cadenas

Una cadena dispone de métodos que permiten obtener nuevas cadenas transformadas.

Por ejemplo:

```
saludo.lower()
```

devuelve una versión en minúsculas.

Es importante observar que:

```
print(saludo.lower())
print(saludo)
```

no modifica el contenido original de `saludo`.

Para conservar el nuevo valor debemos asignarlo:

```
saludo = saludo.lower()
```

Algo parecido sucede con `replace()`:

```
saludo.replace("mundo", "universo")
```

devuelve una cadena nueva en la que se ha realizado la sustitución, pero no modifica automáticamente `saludo`.

> **Las cadenas son inmutables**
>
> Una cadena no se modifica internamente. Operaciones como `lower()` o `replace()` producen una **nueva cadena**. Si queremos conservarla, tendremos que asignarla a una variable.

3.4. Acceder a caracteres y obtener la longitud

Podemos acceder a los caracteres de una cadena mediante su posición:

```
saludo = "¡Hola mundo!"
print(saludo[0])
print(saludo[1])
```

Las posiciones comienzan en **0**:

- `saludo[0]` es `¡`.
- `saludo[1]` es `H`.

La función `len()` devuelve el número de caracteres:

```
len(saludo)
```

Aunque en el comentario original se compara un `str` con un array o una lista, es más preciso decir que una cadena es una **secuencia de caracteres**. Podemos acceder a sus posiciones de forma parecida a una lista, pero una cadena no se puede modificar carácter a carácter porque es inmutable.

3.5. Buscar texto con `in` y `not in`

El operador `in` permite comprobar si un texto aparece dentro de otro:

```
if busca.lower() in texto.lower():
```

En el ejemplo convertimos ambas cadenas a minúsculas antes de compararlas. De esta forma no importa que una aparezca como `"FREE"` y otra como `"free"`.

`not in` comprueba lo contrario:

```
if busca not in texto:
```

> **Atención al segundo ejemplo**
>
> El comentario indica que esta segunda búsqueda tampoco distingue mayúsculas y minúsculas, pero el código **sí las distingue**, porque compara directamente `busca` con `texto`.
>
> Si quisiéramos una búsqueda negativa sin distinguir mayúsculas y minúsculas, podríamos escribir:
>
> ```
> if busca.lower() not in texto.lower():
>     print(f"{busca} NO está en {texto}")
> ```

3.6. Concatenar cadenas

El operador `+` une cadenas:

```
nombre = "Raquel"
apellidos = "García Ross"

print(nombre + " " + apellidos)
```

También podemos construir y guardar una nueva cadena:

```
nombreCompleto = apellidos + ", " + nombre
```

El resultado sería:

```
García Ross, Raquel
```

3.7. Extraer partes de una cadena: slicing

Python permite obtener fragmentos mediante la sintaxis:

```
cadena[inicio:fin:paso]
```

Por ejemplo:

```
saludo = "Hola Mundo"

print(saludo[:4])
print(saludo[5:])
print(saludo[::2])
```

- `saludo[:4]` toma desde el principio hasta la posición 4, sin incluirla.
- `saludo[5:]` toma desde la posición 5 hasta el final.
- `saludo[::2]` recorre la cadena de dos en dos.

También podemos utilizar un paso negativo:

```
saludo[::-1]
```

Esto recorre la cadena en sentido contrario y permite invertirla.

3.8. Recorrer una cadena con `for`

Una cadena puede recorrerse carácter a carácter:

```
for x in nombre:
    print(x)
```

En cada vuelta del bucle, `x` contiene uno de los caracteres.

También podemos recorrer las posiciones:

```
for i in range(len(nombre)):
    print(f"{i}: {nombre[i]}")
```

Aquí se combinan tres elementos:

- `len(nombre)` obtiene la longitud.
- `range(...)` genera las posiciones desde `0`.
- `nombre[i]` obtiene el carácter situado en cada posición.

Por ejemplo:

```
range(5)
```

genera los valores:

```
0
1
2
3
4
```

El `5` no se incluye.

3.9. Posibles mejoras

- Utilizar nombres en estilo `snake_case`, por ejemplo `nombre_completo` en lugar de `nombreCompleto`.
- Corregir el comentario de la segunda búsqueda con `not in`, ya que el código mostrado sí distingue mayúsculas y minúsculas.
- Hablar de las cadenas como **secuencias de caracteres** en vez de considerarlas arrays o listas.
- Cuando solo queremos recorrer los caracteres, `for caracter in cadena` es normalmente más sencillo que usar posiciones.

2.1.4 4. Variables lógicas o booleanas

El tipo `bool` representa valores lógicos. Solo existen dos:

```
True
False
```

4.1. Código completo

```
# lógica o booleana
autorizado = False    # el otro valor posible True
print( type(autorizado)  )


# ejemplo de uso de vaiable lógica
if autorizado==True:  # autorizado==True <=> autorizado
    print("Puede pasar")
    print("resto del bloque positivo")
else:
    print("No puede pasar")
    print("resto del bloque negativo")
```

4.2. El tipo `bool`

La variable:

```
autorizado = False
```

contiene un valor booleano.

Podemos comprobar su tipo:

```
print(type(autorizado))
```

4.3. Utilizar un booleano en una condición

El ejemplo utiliza:

```
if autorizado == True:
```

Si la condición es verdadera, se ejecutan las instrucciones indentadas debajo del `if`. En caso contrario, se ejecuta el bloque asociado a `else`.

Como `autorizado` vale `False`, se ejecutará:

```
print("No puede pasar")
print("resto del bloque negativo")
```

La indentación indica qué instrucciones forman parte de cada bloque.

4.4. Una forma más sencilla

No es necesario comparar explícitamente un booleano con `True`.

En lugar de:

```
if autorizado == True:
```

podemos escribir:

```
if autorizado:
```

Ambas formas expresan la misma condición, pero la segunda es más habitual y clara en Python.

4.5. Posibles mejoras

El ejemplo podría escribirse de forma más sencilla así:

```
autorizado = False

if autorizado:
    print("Puede pasar")
    print("Resto del bloque positivo")
else:
    print("No puede pasar")
    print("Resto del bloque negativo")
```

2.1.5 5. Conversión de tipos

Los datos introducidos mediante `input()` se obtienen como cadenas de texto. Si queremos realizar operaciones numéricas con ellos, tendremos que convertirlos.

5.1. Código completo

```
# conversión de tipos

# Ejemplo sin cast que no "funciona"
saldo = input("saldo: ")
incremento = input("incremento: ")

total = saldo + incremento
print("Valor de total sin cast: ", total)
print(type(saldo))
print(type(incremento))
"""
 El operador + en cadenas concatena
"""

# Se intenta ahora haciendo cast de las cadenas
saldo = float(saldo)
incremento = float(incremento)
total = saldo + incremento
print("Valor de total: ", total)

"""
 => Convertir a números las lecturas antes de operar con ellas
"""
```

5.2. `input()` devuelve una cadena

Estas instrucciones leen datos escritos por el usuario:

```
saldo = input("saldo: ")
incremento = input("incremento: ")
```

Aunque escribamos números, el resultado de `input()` es de tipo `str`.

Por ejemplo, si escribimos:

```
saldo: 100
incremento: 25
```

internamente tendremos dos cadenas equivalentes a:

```
saldo = "100"
incremento = "25"
```

5.3. ¿Qué sucede al utilizar `+` ?

La operación:

```
total = saldo + incremento
```

no suma matemáticamente los valores. Como ambos son cadenas, `+` las **concatena**.

Con los valores anteriores obtendríamos:

```
10025
```

en vez de:

```
125
```

Las llamadas a `type()` permiten comprobar el motivo:

```
print(type(saldo))
print(type(incremento))
```

5.4. Convertir a `float`

Podemos convertir ambas cadenas a números decimales:

```
saldo = float(saldo)
incremento = float(incremento)
```

A partir de ese momento:

```
total = saldo + incremento
```

sí realiza una suma numérica.

Este proceso de convertir un valor de un tipo a otro suele denominarse **conversión de tipos** o, habitualmente, *casting*.

Una forma habitual

Si sabemos desde el principio que necesitamos un número, también podemos convertir directamente el resultado de `input()` :

```
saldo = float(input("Saldo: "))
incremento = float(input("Incremento: "))
```

5.5. ¿Y si el usuario no introduce un número?

La conversión:

```
float(saldo)
```

solo funciona si el contenido puede interpretarse como un número.

Por ejemplo, si el usuario introduce:

```
hola
```

Python producirá un `ValueError` .

Más adelante podremos controlar este tipo de situaciones validando los datos introducidos o tratando las excepciones.

2.2 Estructuras control

2.2.1 Introducción a Python

Las **estructuras selectivas** permiten que un programa tome decisiones y ejecute unas instrucciones u otras dependiendo de si se cumple una condición.

En Python utilizamos principalmente:

- `if`
- `if ... else`
- `if ... elif ... else`
- `match ... case`

1. Código completo

```python
## Selectiva simple (y  probar indentación)

edad = int(input("Edad: "))
print( type(edad) )

if  edad >= 18:  # selectiva simple
    print("Adulto")
    print("puede jugar") ## Error indentación
print("¡Hola Clase!")  # siempre,fuera if




# Ejemplo selectiva if-else y condición lógica con and

saldo = int(input("saldo: "))
edad = int(input("edad: "))

if (edad >= 18 and saldo > 0) or (saldo > 100000):
    print("Puede jugar")
else:
    print("no puede jugar")
print("Fin selectiva")  # siempre se muestra,fuera selectiva


# Ejemplo selectivas anidadas
a = int(input("valor de a:"))
b = 200

if b > a:
    print(f"{b} es mayor que {a}")
else:
    if b < a:
        print(f"{a} es mayor que {b}")
    else:
        print(f"{a} es igual a {b}")

# lo mismo con elif (else if), más legible/compacto
if b > a:
    print(f"{b} es mayor que {a}")
elif b < a:
    print(f"{a} es mayor que {b}")
else:
    print(f"{a} es igual a {b}")


"""
Selector multiple
 Cuando se quieren por ejemplo evaluar las opciones de un menú
 evita concatenar demasiados elif que hacen menos legible el código,
"""

opcion = int(input("Seleccione una opción: "))

match opcion:
    case 0:
        print("Ha seleccionado opción 0")
        print("procedemos opción 0")
    case 1:
        print("Ha seleccionado opción 1")
        print("procedemos opción 1")

    # otros case...

    case _:   #este es el caso por defecto
        print("Opción por defecto")
        print("No ha indicado ninguna opción contemplada")
```

2. Selectiva simple: `if`

Comenzamos leyendo la edad del usuario:

```python
edad = int(input("Edad: "))
print(type(edad))
```

Como `input()` devuelve una cadena, utilizamos `int()` para convertir el dato introducido a un número entero.

Después aparece la estructura:

```python
if edad >= 18:
    print("Adulto")
    print("puede jugar")
```

La expresión:

```
edad >= 18
```

es una **condición**. Su resultado solo puede ser `True` o `False`.

Si la edad es mayor o igual que 18, se ejecutan las instrucciones indentadas debajo del `if`.

Si no se cumple la condición, Python las salta y continúa ejecutando el programa.

2.1. LA IMPORTANCIA DE LA INDENTACIÓN

Observa:

```
if edad >= 18:
    print("Adulto")
    print("puede jugar")

print("¡Hola Clase!")
```

Las dos primeras llamadas a `print()` pertenecen al `if` porque están indentadas.

En cambio:

```
print("¡Hola Clase!")
```

está fuera del bloque, por lo que se ejecutará **siempre**, independientemente de la edad introducida.

> **La indentación delimita los bloques**
>
> En Python, las instrucciones que pertenecen a un `if`, `else`, bucle u otra estructura deben aparecer indentadas.

> **Sobre el comentario del ejemplo**
>
> En el script aparece el comentario `## Error indentación` junto a `print("puede jugar")`, pero esa línea tiene la misma indentación que la instrucción anterior y, tal como está escrita, pertenece correctamente al bloque `if`.
>
> Un error se produciría, por ejemplo, si una de las instrucciones tuviese una indentación incoherente.

3. `if-else`: elegir entre dos alternativas

Cuando queremos ejecutar una acción si se cumple una condición y otra diferente si no se cumple, utilizamos `if ... else`.

```
saldo = int(input("saldo: "))
edad = int(input("edad: "))

if (edad >= 18 and saldo > 0) or (saldo > 100000):
    print("Puede jugar")
else:
    print("no puede jugar")

print("Fin selectiva")
```

La estructura general es:

```
if condicion:
    # instrucciones si la condición es verdadera
else:
    # instrucciones si la condición es falsa
```

Solo se ejecutará uno de los dos bloques.

Después de ambos aparece:

```
print("Fin selectiva")
```

Como no está indentado dentro del `if` ni del `else`, se ejecuta siempre.

4. Condiciones compuestas con `and` y `or`

La condición anterior es más compleja:

```
(edad >= 18 and saldo > 0) or (saldo > 100000)
```

Aquí aparecen dos operadores lógicos.

4.1. OPERADOR `and`

Con `and`, las dos condiciones deben ser verdaderas:

```
edad >= 18 and saldo > 0
```

Se cumple solamente cuando:

- la persona tiene al menos 18 años;
- **y además** su saldo es mayor que 0.

4.2. OPERADOR `or`

Con `or`, basta con que se cumpla una de las condiciones.

La condición completa puede entenderse así:

```
(edad mínima Y saldo positivo)
O
(saldo mayor que 100000)
```

Por tanto, el programa permitirá continuar si se cumple cualquiera de esos dos grupos de condiciones.

> **Usar paréntesis ayuda a leer**
>
> Aunque en algunos casos Python podría interpretar correctamente la expresión sin paréntesis, utilizarlos ayuda a ver claramente qué condiciones se están agrupando.

5. Selectivas anidadas

Una estructura selectiva puede contener otra estructura selectiva.

El ejemplo compara dos números:

```
a = int(input("valor de a:"))
b = 200

if b > a:
    print(f"{b} es mayor que {a}")
else:
    if b < a:
        print(f"{a} es mayor que {b}")
    else:
        print(f"{a} es igual a {b}")
```

Primero se comprueba:

```
b > a
```

Si se cumple, ya sabemos que `b` es mayor.

Si no se cumple, entramos en el `else` y hacemos otra comprobación:

```
b < a
```

Si tampoco se cumple esta segunda condición, solo queda una posibilidad: ambos números son iguales.

Este tipo de construcción se denomina **selectiva anidada** porque hay un `if` dentro de otro bloque.

6. Simplificar con `elif`

Python dispone de `elif`, abreviatura de *else if*.

El ejemplo anterior puede escribirse así:

```
if b > a:
    print(f"{b} es mayor que {a}")
elif b < a:
    print(f"{a} es mayor que {b}")
else:
    print(f"{a} es igual a {b}")
```

Esta versión expresa exactamente las mismas posibilidades:

1. `b` es mayor que `a`.
2. `b` es menor que `a`.
3. Ambos son iguales.

Normalmente resulta más clara y evita añadir niveles innecesarios de indentación.

Python comprueba las condiciones de arriba hacia abajo y ejecuta el bloque correspondiente a la **primera que sea verdadera**.

Si ninguna se cumple, ejecuta el `else`.

7. Selector múltiple con `match`

Cuando queremos comparar un valor con varias opciones concretas podemos utilizar `match`.

```
opcion = int(input("Seleccione una opción: "))

match opcion:
    case 0:
        print("Ha seleccionado opción 0")
        print("procedemos opción 1")

    case 1:
        print("Ha seleccionado opción 1")
        print("procedemos opción 2")

    case _:
        print("Opción por defecto")
        print("No ha indicado ninguna opción contemplada")
```

Python toma el valor almacenado en `opcion` y busca un `case` que coincida.

Si el usuario introduce `0`, se ejecuta `case 0`. Si introduce `1`, se ejecuta `case 1`.

Para cualquier otro valor se ejecuta:

```
case _:
```

El guion bajo `_` representa aquí el **caso por defecto**.

7.1. `match` FRENTE A MUCHOS `elif`

Un menú también podría escribirse utilizando varios `elif`:

```
if opcion == 0:
    ...
elif opcion == 1:
    ...
elif opcion == 2:
    ...
else:
    ...
```

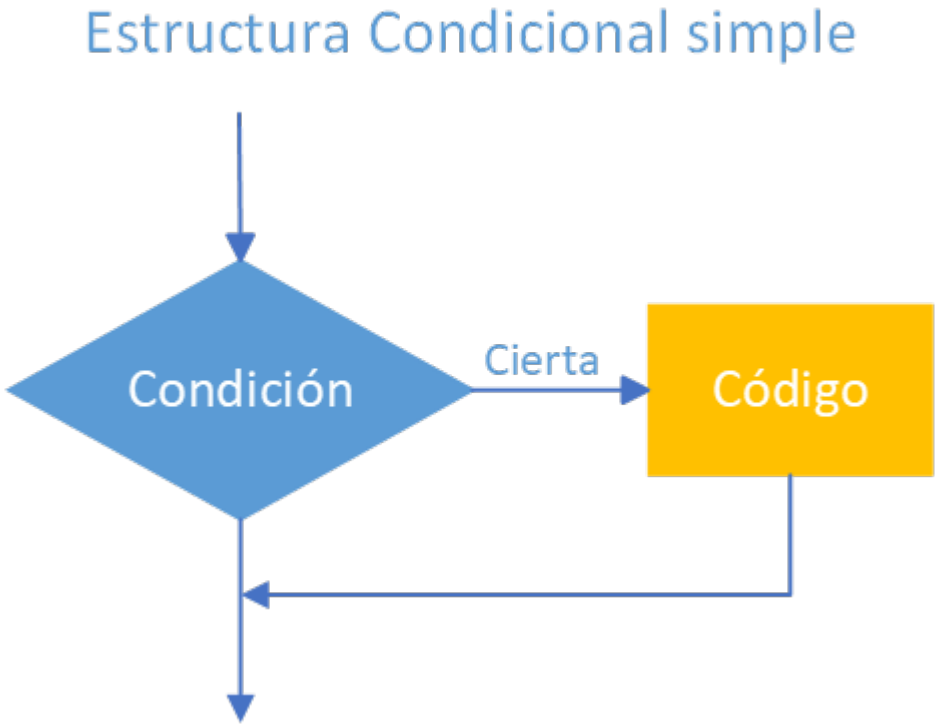
Cuando tenemos muchas opciones concretas, `match` puede resultar más legible.

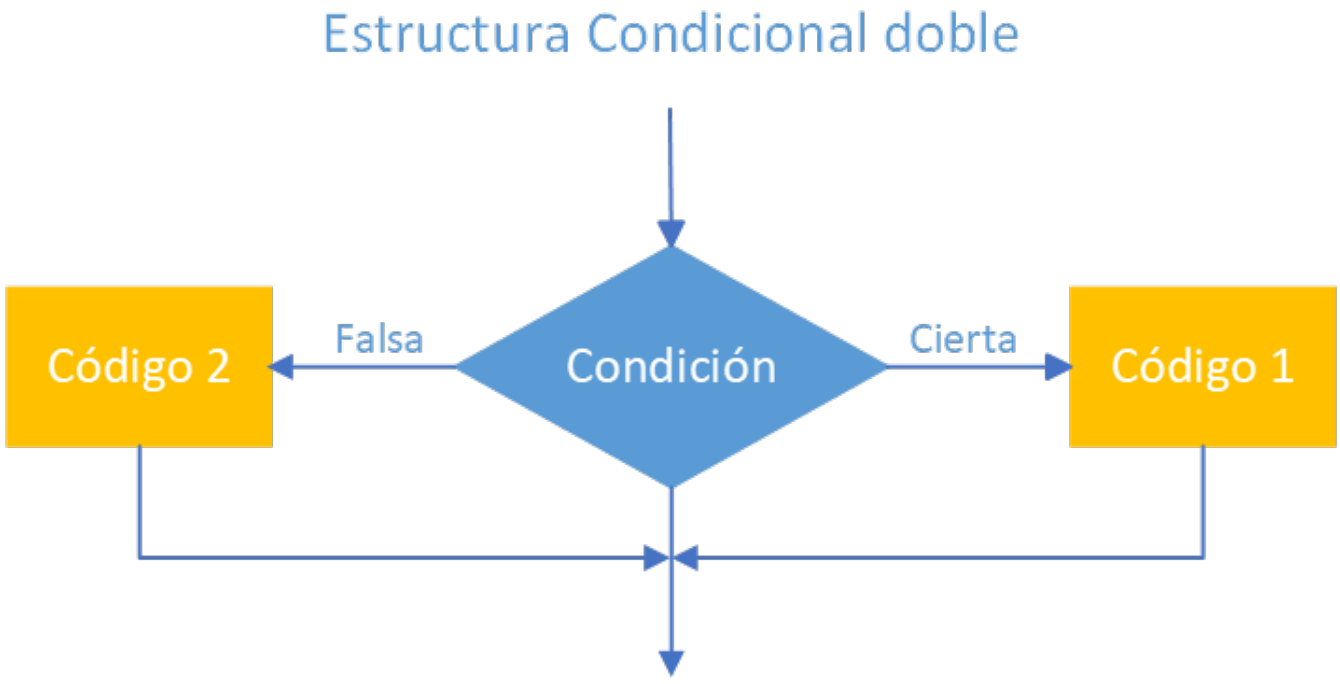
Versión de Python

La sentencia `match` está disponible a partir de **Python 3.10**.

8. Anexo enlaces y figuras

https://www.w3schools.com/python/python_conditions.asp





2.2.2 Introducción a los bucles en Python

Los **bucles** permiten repetir un conjunto de instrucciones varias veces.

En Python utilizaremos principalmente dos tipos:

- `for`, cuando queremos recorrer una secuencia o repetir algo un número conocido de veces.
- `while`, cuando queremos repetir mientras se cumpla una condición.

En estos ejemplos también veremos:

- `range()`
- acumuladores
- `break`
- recorrido de cadenas de texto

1. Código completo

```
# bucle for básico

# con índices y uso range
for i in range(5):
    print(i)

for i in range(10, 20):
    print(f"{i} ", end=" ")

# Suma números
suma = 0  # acumulador
ini = int(input("inicio: "))
fin = int(input("fin: "))

for i in range(ini, fin+1):
    print(i)
    suma += i  # equivale a suma = suma + i

print("\nSuma:", suma)  # \n salto de línea

# Bucle while

numero = int(input("Introduce un número (0 para terminar): "))
while numero != 0:
    print("Has introducido:", numero)
    numero = int(input("Introduce otro número (0 para terminar): "))
print("Fin del programa")

while True:  # el uso del break debe ser "cuidadoso..."
    numero = int(input("Introduce un número (0 para terminar): "))
    if numero == 0:
        break
    print("Has introducido:", numero)
print("Fin del programa")

# Se muestra ahora el uso de cadenas con bucle for

nombre = "www.aulas.local"
# a) Recorrer cada caracter de un str
for x in nombre:
    print(x)

# b) Recorrer cada caracter de un str
# usando el índice o posición si es necesario en el bucle
for i in range(len(nombre)):
    print(f"Pos{i}: {nombre[i]}")
```

2. Bucle `for` básico

Un bucle `for` permite repetir instrucciones recorriendo una secuencia de valores.

El primer ejemplo es:

```
for i in range(5):
    print(i)
```

`range(5)` genera los valores:

```
0
1
2
3
4
```

En cada vuelta del bucle, la variable `i` toma uno de esos valores.

Por tanto, la salida será:

```
0
1
2
3
4
```

> **El valor final no se incluye**
>
> `range(5)` genera valores desde `0` hasta `4`.
>
> El `5` no forma parte del recorrido.

3. Indicar inicio y fin con `range()`

También podemos indicar dónde empieza y dónde termina el recorrido:

```
for i in range(10, 20):
    print(f"{i} ", end=" ")
```

En este caso, `range(10, 20)` genera:

```
10 11 12 13 14 15 16 17 18 19
```

De nuevo, el límite final no se incluye.

3.1. EL PARÁMETRO `end` DE `print()`

Normalmente, `print()` termina con un salto de línea.

Por ejemplo:

```
print(10)
print(11)
```

produce:

```
10
11
```

Sin embargo:

```
print(f"{i} ", end=" ")
```

hace que cada valor se muestre en la misma línea.

El argumento `end` indica qué debe escribirse después de cada `print()`.

4. Utilizar un acumulador

El siguiente ejemplo suma todos los números comprendidos entre dos valores introducidos por el usuario:

```
suma = 0
ini = int(input("inicio: "))
fin = int(input("fin: "))
```

```
for i in range(ini, fin + 1):
    print(i)
    suma = suma + i

print("\nSuma:", suma)
```

La variable:

```
suma = 0
```

se utiliza como **acumulador**.

Un acumulador es una variable que va guardando un resultado parcial y se actualiza en cada vuelta del bucle.

En este caso:

```
suma = suma + i
```

significa:

1. tomar el valor actual de `suma`;
2. añadirle el valor de `i`;
3. guardar el nuevo resultado otra vez en `suma`.

4.1. EJEMPLO

Si el usuario introduce:

```
inicio: 1
fin: 4
```

el bucle recorrerá:

```
1
2
3
4
```

y el acumulador evolucionará así:

```
suma = 0
suma = 0 + 1
suma = 1 + 2
suma = 3 + 3
suma = 6 + 4
```

El resultado final será:

```
10
```

4.2. ¿POR QUÉ APARECE `fin + 1`?

El código utiliza:

```
range(ini, fin + 1)
```

porque `range()` no incluye el límite final.

Si queremos incluir el valor almacenado en `fin`, debemos utilizar `fin + 1`.

Forma abreviada

La instrucción:

```
suma = suma + i
```

también puede escribirse:

```
suma += i
```

Ambas hacen lo mismo.

5. Bucle `while`

Un bucle `while` repite instrucciones **mientras una condición sea verdadera**.

El ejemplo es:

```
numero = int(input("Introduce un número (0 para terminar): "))

while numero != 0:
    print("Has introducido:", numero)
    numero = int(input("Introduce otro número (0 para terminar): "))

print("Fin del programa")
```

La condición es:

```
numero != 0
```

El bucle continuará mientras el usuario introduzca un valor distinto de cero.

Cuando introduzca:

```
0
```

la condición pasará a ser falsa y el programa saldrá del bucle.

Después se ejecutará:

```
print("Fin del programa")
```

6. Actualizar la condición del `while`

Hay un detalle especialmente importante:

```
numero = int(input("Introduce otro número (0 para terminar): "))
```

Esta instrucción aparece **dentro del bucle**.

Gracias a ella, `numero` recibe un nuevo valor en cada iteración y la condición:

```
numero != 0
```

puede llegar a ser falsa.

> ⚠ **Cuidado con los bucles infinitos**
>
> Si olvidásemos actualizar `numero`, la condición podría permanecer siempre verdadera y el bucle no terminaría.
>
> Por ejemplo:
>
> ```
> numero = 5
>
> while numero != 0:
>     print(numero)
> ```
>
> Este código imprimiría `5` continuamente.

7. `while True` y `break`

El siguiente ejemplo realiza prácticamente la misma tarea de otra forma:

```
while True:
    numero = int(input("Introduce un número (0 para terminar): "))

    if numero == 0:
        break

    print("Has introducido:", numero)

print("Fin del programa")
```

La condición:

```
while True:
```

es siempre verdadera.

Por tanto, en principio, este bucle no terminaría nunca.

Sin embargo, dentro aparece:

```
if numero == 0:
    break
```

La instrucción `break` provoca la **salida inmediata del bucle**.

Cuando el usuario introduce `0`, el programa abandona el `while` y continúa después de él.

7.1. DOS FORMAS DE RESOLVER EL MISMO PROBLEMA

Primera forma:

```
while numero != 0:
```

Segunda forma:

```
while True:
    ...
    if numero == 0:
        break
```

Ambas pueden ser válidas.

Para ejemplos sencillos, la primera suele expresar de forma muy clara la condición de repetición.

La segunda puede resultar útil cuando la condición para terminar aparece de forma natural dentro del bucle.

> ⚠ **Usar `break` con criterio**
>
> `break` no es incorrecto, pero conviene utilizarlo cuando realmente mejora la claridad del programa.
>
> Un exceso de salidas repartidas por un bucle puede hacer que el código sea más difícil de seguir.

8. Recorrer una cadena con `for`

Una cadena de texto puede recorrerse carácter a carácter.

Por ejemplo:

```
nombre = "www.aulas.local"

for x in nombre:
    print(x)
```

En cada iteración, `x` contiene uno de los caracteres de la cadena.

La salida comenzaría así:

```
w
w
w
.
a
u
...
```

Esta es normalmente la forma más sencilla de recorrer una cadena cuando solo necesitamos sus caracteres.

9. Recorrer una cadena utilizando índices

También podemos recorrer las posiciones de la cadena:

```
for i in range(len(nombre)):
    print(f"{i}: {nombre[i]}")
```

Aquí se combinan varias ideas.

9.1. `len(nombre)`

Obtiene la longitud de la cadena.

Si:

```
nombre = "www.aulas.local"
```

`len(nombre)` devuelve el número de caracteres que contiene.

9.2. `range(len(nombre))`

Genera todas las posiciones válidas:

```
0
1
2
3
...
```

9.3. `nombre[i]`

Permite obtener el carácter situado en la posición `i`.

Por tanto, el programa mostrará algo similar a:

```
0: w
1: w
2: w
```

```
3: .
4: a
...
```

10. ¿Cuándo usar cada forma?

Si solo queremos los caracteres:

```
for caracter in nombre:
    print(caracter)
```

suele ser la opción más sencilla.

Si necesitamos conocer también la posición de cada carácter:

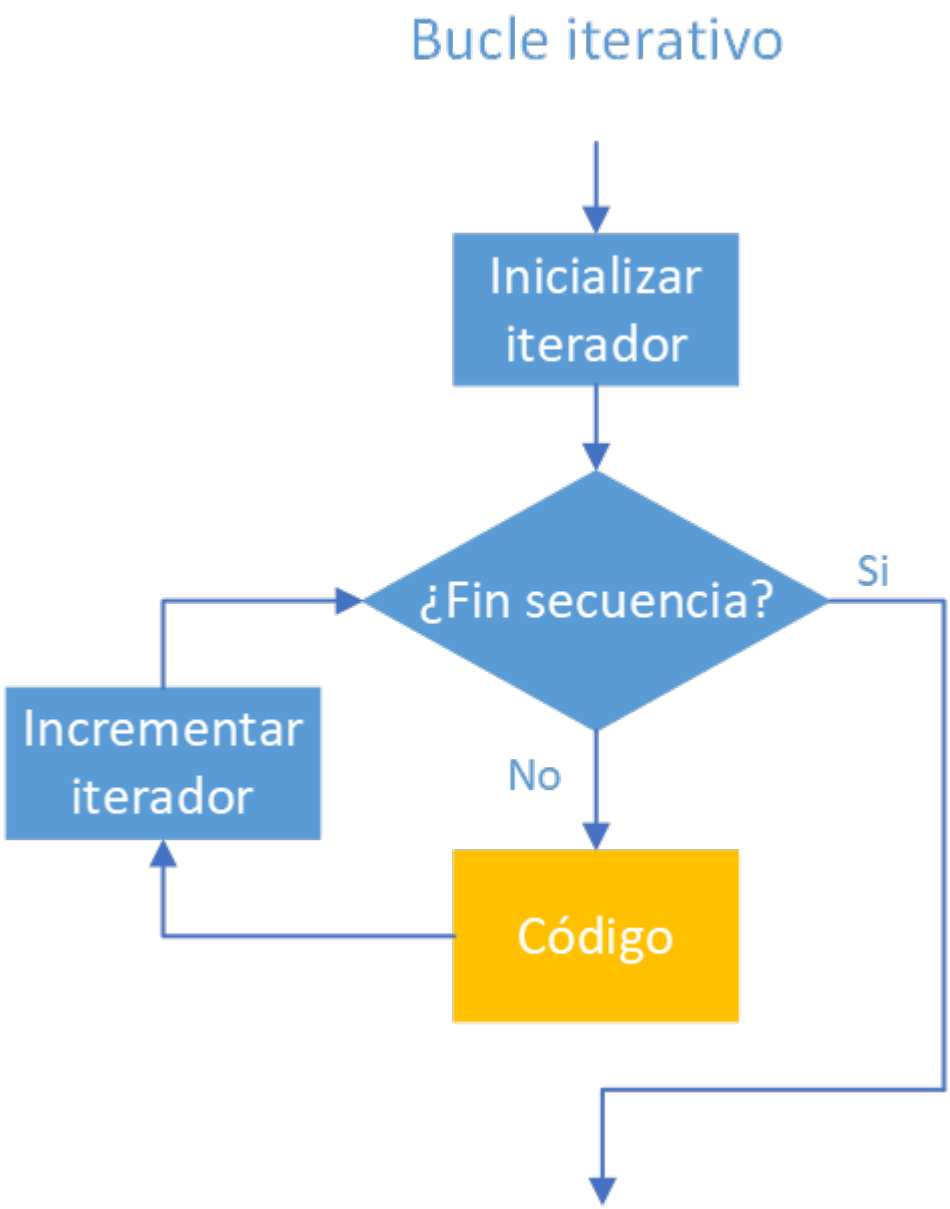
```
for i in range(len(nombre)):
    print(i, nombre[i])
```

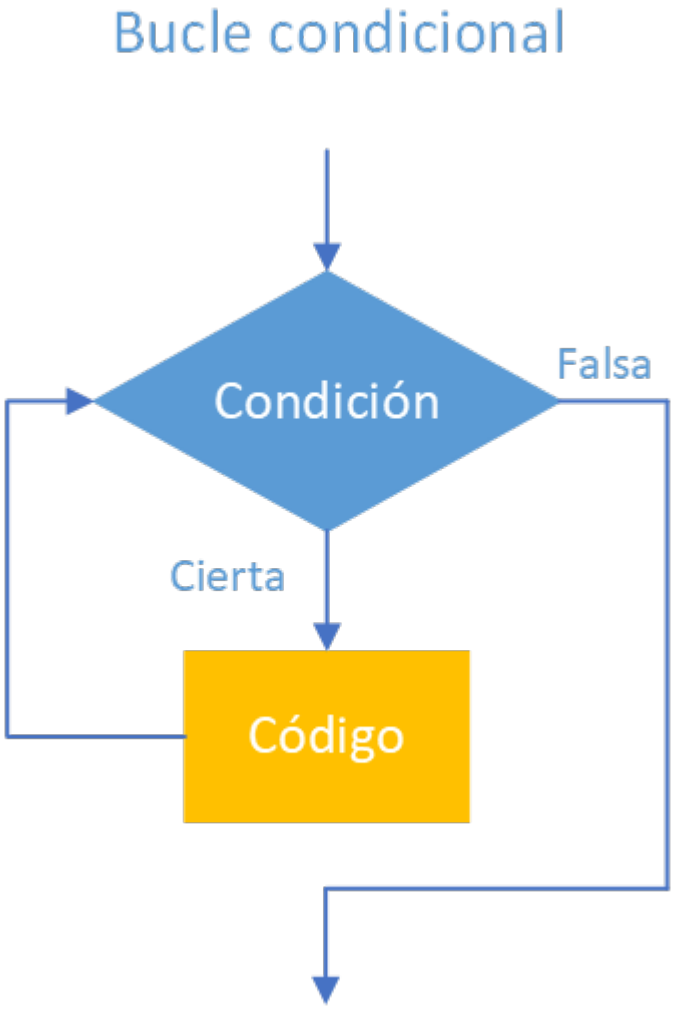
podemos recorrer los índices.

Más adelante veremos que Python también ofrece otras formas cómodas de obtener simultáneamente posición y valor.

11. Enlaces y figuras

- https://www.w3schools.com/python/python_for_loops.asp
- https://www.w3schools.com/python/python_while_loops.asp


Bucle iterativo
Inicializar iterador
¿Fin secuencia?
Si
No
Incrementar iterador
Código


Bucle condicional
Condición
Falsa
Cierta
Código

2.3 Estructuras Datos

2.3.1 Listas

Las **listas** permiten almacenar varios valores dentro de una misma variable.

Por ejemplo:

```
puertos = [22, 80, 443, 8080]
```

Una lista puede contener números, cadenas u otros tipos de datos. En estos ejemplos veremos cómo crear listas, acceder a sus elementos, obtener su longitud, extraer partes mediante *slicing*, modificar elementos, comprobar si una lista está vacía, recorrer listas, buscar elementos, ordenarlas y convertir cadenas en listas.

También aparecen dos contenidos algo más avanzados: las **comprensiones de listas** y el uso de `try/except`.

1. Crear listas y acceder a sus elementos

1.1. CÓDIGO COMPLETO

```python
# Definir lista y uso básico

# Lista de enteros
puertos = [22, 80, 443, 8080]

# mostar la lista y su tipo
print(puertos)
print(type(puertos))

# mostrar un elemento de la lista
print(puertos[0])
print(puertos[1])

# mostrar el tamaño (longitud => len())
print("Nº de elementos de la lista:", len(puertos))


# Ejemplos uso de lista: mostrar un rango
frutas = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print("frutas", frutas)
print("frutas[1:4]", frutas[1:4])

# Ejemplos uso de lista: generar nueva lista a partir de un rango
nueva_lista = frutas[1:4]
print("nueva_lista", nueva_lista)

# imprimir el último de una lista: usando len o con índice negativo...
print("frutas[len(frutas)-1]", frutas[len(frutas)-1])
print("Elemento finl: ", frutas[-1])

# modificar un elemento de la lista
print("Elemento 0: ", frutas[0])
print('ejecutamos frutas[0]="watermelon"')
frutas[0]="watermelon"
print("Elemento 0: ", frutas[0])


# Verificar lista vacía
lista = [1,2]
print("lista: ",lista)
print("len(lista):", len(lista))

# B) usando longitud
if len(lista) != 0:
    print("La lista contiene elementos")
else:
    print("Lista vacía")

# A) preferible usando la lista como condición lógica
lista =  []
print("lista: ",lista)
print("len(lista):", len(lista))
if lista: # if len(lista) != 0:
    print("La lista contiene elementos")
else:
    print("Lista vacía")


# Ejemplos formas de recorrer lista
lista_IP = ["192.168.1.1", "10.10.10.10", "172.16.100.100", "195.172.1.1"]

# los elementos con for
for x in lista_IP:
    print("IP: ", x)

# los elementos y sus índices con for
for i in range(len(lista_IP)):
    print(f"IP[{i}]: {lista_IP[i]}")

# Avanzado: comprensión de listas: [expresión for elemento in iterable]
[print(" -", x) for x in lista_IP ]
```

1.2. CREAR UNA LISTA

Una lista se escribe entre corchetes:

```python
puertos = [22, 80, 443, 8080]
```

En este caso contiene cuatro números enteros. Podemos mostrarla completa con `print(puertos)` y comprobar su tipo con `type(puertos)`.

1.3. ACCEDER A ELEMENTOS MEDIANTE SU POSICIÓN

Los elementos se numeran empezando en `0`:

```
print(puertos[0])
print(puertos[1])
```

> **Los índices comienzan en 0**
>
> El primer elemento de una lista está en la posición `0`, no en la `1`.

1.4. OBTENER LA LONGITUD CON `len()`

```
len(puertos)
```

devuelve el número de elementos de la lista.

1.5. OBTENER UNA PARTE DE UNA LISTA

Podemos utilizar *slicing*:

```
  frutas[1:4]
```

La posición inicial se incluye y la final no. También podemos guardar el fragmento obtenido en una nueva lista:

```
nueva_lista =   frutas[1:4]
```

1.6. ACCEDER AL ÚLTIMO ELEMENTO

Podemos calcular su posición:

```
  frutas[len(  frutas) - 1]
```

pero normalmente es más sencillo utilizar un índice negativo:

```
  frutas[-1]
```

Los índices negativos cuentan desde el final: `-1` es el último elemento, `-2` el penúltimo, etc.

1.7. MODIFICAR ELEMENTOS

Las listas son **mutables**, por lo que podemos modificar sus elementos directamente:

```
  frutas[0] = "watermelon"
```

> **Listas y cadenas no se comportan igual**
>
> Las cadenas son inmutables, pero las listas sí permiten modificar sus elementos.

2. Comprobar si una lista está vacía

Podemos hacerlo comprobando su longitud:

```
if len(lista) != 0:
    print("La lista contiene elementos")
else:
    print("Lista vacía")
```

Python permite una forma más sencilla:

```
if lista:
    print("La lista contiene elementos")
```

```
else:
    print("Lista vacía")
```

Una lista vacía (`[]`) se considera falsa en una condición; una lista con elementos se considera verdadera.

3. Recorrer una lista

Si solo necesitamos los valores:

```
for x in lista_IP:
    print("IP: ", x)
```

Si necesitamos también las posiciones:

```
for i in range(len(lista_IP)):
    print(f"IP[{i}]: {lista_IP[i]}")
```

3.1. COMPRENSIÓN DE LISTAS

El script incluye:

```
[print(" -", x) for x in lista_IP]
```

La sintaxis general de una comprensión es:

```
[expresion for elemento in iterable]
```

> **Contenido avanzado**
>
> Aunque esta construcción funciona, las comprensiones de listas se utilizan sobre todo para **crear nuevas listas**, no para ejecutar acciones como `print()`.
>
> Para mostrar elementos es más claro:
>
> ```
> for x in lista_IP:
>     print(" -", x)
> ```

4. Buscar elementos en una lista

4.1. CÓDIGO COMPLETO

```python
# BSUCAR elemento en la lista

# Con in => True/False

lista_IP = ["192.168.1.1", "10.10.10.10", "172.16.100.100", "195.172.1.1"]
print("lista_IP", lista_IP)

ip = input("IP a buscar: ")
if ip in lista_IP:
    print(f"Yes, {ip} is in {lista_IP}")
else:
    print(f"No, {ip} is not in {lista_IP}")

# Buscar con index si necesitamos la posición
# Nota: index genera excepción si no está

print("lista_IP", lista_IP)
ip = input("IP a buscar: ")

# pos = lista_IP.index(ip)  # Genera excepción si no está en la lista

# Para corregir excepción si no se encuentra
# a) usar in o count antes de index
if ip in lista_IP:
    pos = lista_IP.index(ip)
    print(f"{ip} está en la posición {pos}")
else:
    print(f"{ip} NO está en {lista_IP}")

# b) se añade bloque try/except  para evitarlo
try:
    pos = lista_IP.index(ip)
    print(f"{ip} está en la posición {pos}")
except ValueError:
    print(f"{ip} NO está en {lista_IP}")
```

4.2. BUSCAR CON `in`

La forma más sencilla de comprobar si un elemento existe es:

```python
if ip in lista_IP:
```

`in` produce `True` si el elemento está en la lista y `False` si no está.

4.3. OBTENER LA POSICIÓN CON `index()`

```python
lista_IP.index(ip)
```

devuelve la posición del elemento.

Si el elemento no existe, Python genera una excepción `ValueError`. Por eso en el ejemplo se comprueba primero con `in`.

4.4. ALTERNATIVA CON `try/except`

```python
try:
    pos = lista_IP.index(ip)
    print(f"{ip} está en la posición {pos}")
except ValueError:
    print(f"{ip} NO está en {lista_IP}")
```

Contenido algo más avanzado

`try/except` sirve para controlar excepciones durante la ejecución. Para una primera aproximación a listas, comprobar antes con `in` resulta más sencillo.

5. Ordenar y modificar listas

5.1. CÓDIGO COMPLETO

```python
# ordenar listas: sort ,reverse
edades = [19, 17, 23, 56, 48, 100, 31]
print("edades:", edades)
```

```
edades.sort()
print("edades tras edades.sort():", edades)
edades.reverse()
print("edades tras edades.reverse():", edades)

# Modificar la lista: añadir al final
print("edades:", edades)
edades.append(20)
print("edades tras edades.append(20):", edades)

# Modificar la lista: añadir en cualquier posición:
edades.insert(1, 44)
print("edades tras edades.insert(1,44):", edades)

# Borrar un elemento
eliminado = edades.pop()
print("edades tras eliminado = edades.pop()", edades)
print("eliminado:", eliminado )

eliminado = edades.pop(0)
print("edades tras eliminado = edades.pop(0)", edades)
print("eliminado:", eliminado )

# Vaciar la lista
edades.clear()
print("edades tras clear", edades)

## CONVERTIR string en lista: split
users = "ana:luis:alfredo"
lista = users.split(":")
print("users:", users)
print("type(users)", type(users))
print("lista: ", lista)
print("type(lista)", type(lista))

# Ejemplo lectura notas con input y conversión a lista para calcular media
lectura = input("Notas separadas por el caracter ';': " )
print(lectura, type(lectura)) # es un string
notas = lectura.split(";") # es una lista
print(notas, type(notas))

suma = 0 # Almacena la suma de las notas
for x in notas:
    suma = suma + float(x)
media = suma/len(notas)
print("Suma: ", suma)
print(f"Media: {media:.2f}")

# Avanzado: conversión a números de una lista de string
edades = ["14", "21", "17", "33", "45"]
print("type(edades[0]):", type(edades[0]))
# comprensión de listas
edades =  [int(x) for x in edades]
print("type(edades[0]):", type(edades[0]))
```

5.2. ORDENAR CON `sort()`

```
edades.sort()
```

ordena la lista de menor a mayor y modifica la lista original.

5.3. INVERTIR CON `reverse()`

```
edades.reverse()
```

invierte el orden actual.

> **Ordenar e invertir no es lo mismo**
>
> `reverse()` no ordena: solo invierte el orden existente.

5.4. AÑADIR CON `append()`

```
edades.append(20)
```

añade un elemento al final.

5.5. INSERTAR EN UNA POSICIÓN

```
edades.insert(1, 44)
```

inserta `44` en la posición `1`.

5.6. ELIMINAR CON `pop()`

```
eliminado = edades.pop()
```

elimina el último elemento y lo devuelve. También podemos indicar una posición:

```
eliminado = edades.pop(0)
```

5.7. VACIAR UNA LISTA CON `clear()`

```
edades.clear()
```

elimina todos sus elementos.

6. Convertir una cadena en lista con `split()`

Si tenemos:

```
users = "ana:luis:alfredo"
```

podemos dividir la cadena usando `:` como separador:

```
lista = users.split(":")
```

El resultado será:

```
["ana", "luis", "alfredo"]
```

7. Leer varios valores y calcular una media

Si el usuario introduce:

```
5;7.5;8;6
```

la instrucción:

```
notas = lectura.split(";")
```

produce una lista similar a:

```
["5", "7.5", "8", "6"]
```

> **Siguen siendo cadenas**
>
> `split()` crea una lista, pero sus elementos siguen siendo de tipo `str`.

Por eso el programa convierte cada elemento al sumar:

```
for x in notas:
    suma = suma + float(x)
```

Finalmente calcula:

```
media = suma / len(notas)
```

8. Convertir todos los elementos de una lista

Partimos de:

```
edades = ["14", "21", "17", "33", "45"]
```

Con una comprensión:

```
edades = [int(x) for x in edades]
```

se obtiene una nueva lista de enteros.

Contenido avanzado

Una forma equivalente con un `for` tradicional sería:

```
nueva_lista = []

for x in edades:
    nueva_lista.append(int(x))

edades = nueva_lista
```

9. Enlaces

https://www.w3schools.com/python/python_lists.asp

2.3.2 Introducción a Python: Diccionarios

Los **diccionarios** son estructuras de datos que permiten almacenar información utilizando pares de **clave y valor**.

Por ejemplo:

```
coche = {
    "marca": "Seat",
    "modelo": "Ateca",
    "color": "blanco",
    "kilometros": 125000
}
```

En este diccionario:

- `"marca"` es una clave y `"Seat"` es su valor;
- `"modelo"` es una clave y `"Ateca"` es su valor;
- `"color"` es una clave y `"blanco"` es su valor;
- `"kilometros"` es una clave y `125000` es su valor.

Los diccionarios resultan especialmente útiles cuando queremos agrupar varios datos relacionados y acceder a ellos mediante un nombre significativo.

1. Código completo

```python
# Dictionaries are used to store data values in key:value pairs.
# A dictionary is a collection which is ordered*, changeable and do not allow duplicates.

coche = {
    "marca": "Seat",
    "modelo": "Ateca",
    "color": "blanco",
    "kilometros": 125000
}

# muestra el diccionario completo
print("coche:", coche)

# Mostar el valor de una clave: 2 posibilidades
# a) muestra un elemento con notación []
print('coche["color"]:', coche["color"])
print('coche["kilometros"]:', coche["kilometros"])
# b) muestra un elemento con método get
print('coche.get("modelo"):', coche.get("modelo"))


# Recorrer los valores del diccionario y sus claves
print("Bucle for que recorre diccionario")
for clave in coche:
    print(" *", clave, "=>", coche[clave])

# Otra forma de recorrer: items()
# `items()` permite obtener directamente la clave y el valor
#  en cada iteración. Rn muchos casos resulta más cómodo y legible.
print("Bucle for que recorre diccionario con .items()")
for clave, valor in coche.items():
    print(" - ", clave, "=>", valor)

# Modificar dicccionario: dos posibilidades
# a) con notación []
coche["color"] = "gris"
# b) con métoddo update
coche.update({"kilometros": 99999})
print("coche tras modificaciones:", coche)

# Añadir nueva clave a un diccionario
coche["taller"] = "Alterio reparaciones"
print("coche tras añadir clave:", coche)

# uso de values() y keys()
valores = coche.values()
print("valores:", valores)
claves = coche.keys()
print("keys:", claves)

# las variables asignadas con values y keys()
#  se actualizan tras cambios en el diccionario
print('Se añade precio: coche["precio"] = 17000')
coche["precio"] = 17000
print("valores:", valores)
print("keys:", claves)
```

2. Crear un diccionario

Un diccionario se escribe utilizando llaves:

```python
coche = {
    "marca": "Seat",
    "modelo": "Ateca",
    "color": "blanco",
    "kilometros": 125000
}
```

Cada elemento tiene la forma:

```
clave: valor
```

Por ejemplo:

```
"marca": "Seat"
```

La clave es:

```
marca
```

y el valor asociado es:

```
Seat
```

Claves y valores

En una lista accedemos a los elementos normalmente mediante su posición:

```
lista[0]
```

En un diccionario utilizamos una **clave**:

```
coche["marca"]
```

Esto hace que el código pueda resultar más descriptivo.

3. Mostrar el diccionario completo

Podemos mostrar todo el diccionario utilizando `print()`:

```
print("coche:", coche)
```

Obtendremos un resultado similar a:

```
coche: {'marca': 'Seat', 'modelo': 'Ateca', 'color': 'blanco', 'kilometros': 125000}
```

4. Acceder a un valor mediante su clave

Podemos utilizar corchetes:

```
coche["color"]
```

Por ejemplo:

```
print(coche["color"])
```

mostrará:

```
blanco
```

También podemos consultar:

```
coche["kilometros"]
```

y obtendremos:

```
125000
```

La clave nos permite indicar directamente qué dato queremos consultar.

5. Acceder con `get()`

Otra posibilidad es utilizar el método:

```
get()
```

Por ejemplo:

```
coche.get("modelo")
```

devuelve:

```
Ateca
```

Por tanto:

```
print(coche.get("modelo"))
```

es otra forma de consultar un valor.

5.1. DIFERENCIA ENTRE `[]` Y `get()`

Mientras la clave exista, estas dos expresiones devuelven el mismo resultado:

```
coche["modelo"]
```

```
coche.get("modelo")
```

La diferencia aparece cuando la clave **no existe**.

Si intentamos:

```
coche["precio"]
```

antes de haber creado esa clave, Python genera:

```
KeyError
```

Sin embargo:

```
coche.get("precio")
```

devuelve por defecto:

```
None
```

sin detener el programa.

> **`get()` permite indicar un valor por defecto**
>
> Podemos escribir:
>
> ```
> coche.get("precio", 0)
> ```
>
> Si `"precio"` no existe, obtendremos `0`.

6. Recorrer un diccionario con `for`

El ejemplo utiliza:

```
for x in coche:
    print(" *", x, "=>", coche[x])
```

Cuando recorremos directamente un diccionario:

```
for x in coche:
```

la variable `x` va tomando las **claves** del diccionario.

Por ejemplo:

```
marca
modelo
```

```
color
kilometros
```

Podemos utilizar la clave para consultar su valor:

```
coche[x]
```

De esta forma podemos mostrar ambos:

```
marca => Seat
modelo => Ateca
color => blanco
kilometros => 125000
```

7. Modificar un valor

Los diccionarios son **mutables**, por lo que podemos cambiar sus valores.

Por ejemplo:

```
coche["color"] = "gris"
```

La clave `"color"` ya existe, por lo que su valor pasa de:

```
blanco
```

a:

```
gris
```

8. Modificar mediante `update()`

También podemos utilizar:

```
coche.update({"kilometros": 99999})
```

El método `update()` recibe otro diccionario con los datos que queremos actualizar.

Después de ejecutar esa instrucción, el valor asociado a `"kilometros"` será:

```
99999
```

Para modificar un único elemento, tanto:

```
coche["kilometros"] = 99999
```

como:

```
coche.update({"kilometros": 99999})
```

son posibles.

9. Añadir una nueva clave

La misma sintaxis que utilizamos para modificar puede servir para crear un elemento nuevo:

```
coche["taller"] = "Alterio reparaciones"
```

Como la clave `"taller"` no existía, Python la añade al diccionario.

Ahora tendremos también:

```
taller => Alterio reparaciones
```

Modificar o añadir

Esta instrucción:

```
diccionario[clave] = valor
```

hace dos cosas posibles:

- si la clave ya existe, modifica su valor;
- si no existe, crea una nueva entrada.

10. Obtener los valores con `values()`

El método:

```
coche.values()
```

permite obtener una vista de todos los valores del diccionario.

En el ejemplo:

```
valores = coche.values()
print("valores:", valores)
```

aparecerán los valores correspondientes a elementos como:

```
Seat
Ateca
gris
99999
Alterio reparaciones
```

11. Obtener las claves con `keys()`

El método:

```
coche.keys()
```

permite obtener las claves.

Por ejemplo:

```
keys = coche.keys()
print("keys:", keys)
```

incluirá claves como:

```
marca
modelo
color
kilometros
taller
```

12. Las vistas `keys()` y `values()` se actualizan

El ejemplo guarda:

```
valores = coche.values()
keys = coche.keys()
```

Después añade:

```
coche["precio"] = 17000
```

Y vuelve a mostrar:

```
print("valores:", valores)
print("keys:", keys)
```

La nueva clave y el nuevo valor también aparecen.

Esto sucede porque `keys()` y `values()` devuelven **vistas del diccionario**, no copias independientes.

> **Una vista refleja los cambios**
>
> Si el diccionario cambia, las vistas obtenidas con:
>
> ```
> keys()
> values()
> ```
>
> reflejan esos cambios.

13. Otra forma habitual de recorrer un diccionario

Además de:

```
for clave in coche:
    print(clave, coche[clave])
```

existe el método `items()` :

```
for clave, valor in coche.items():
    print(clave, "=>", valor)
```

`items()` permite obtener directamente la clave y el valor en cada iteración. Para muchos casos resulta especialmente cómodo y legible.

14. Enlaces

https://www.w3schools.com/python/python_dictionaries.asp

2.3.3 Estructuras de datos combinadas: listas y diccionarios

Hasta ahora ha trabajado con **listas** y **diccionarios** por separado. Sin embargo, en programas reales es muy habitual combinar ambas estructuras.

En este tema se presentan dos casos frecuentes:

1. un **diccionario que contiene una lista**;
2. una **lista cuyos elementos son diccionarios**.

1. Diccionario con una lista como valor

Un diccionario puede almacenar distintos tipos de valores. entre ellos una lista:

Por ejemplo:

```
servidor = {
    "nombre": "web01",
    "ip": "192.168.1.20",
    "servicios": ["ssh", "http", "https"]
}
```

- `"nombre"` contiene una cadena;
- `"ip"` contiene una cadena;
- `"servicios"` contiene una **lista**.

Esto resulta útil cuando un elemento puede tener varios valores relacionados. En este caso, un servidor puede tener varios servicios activos.

Uan vez definido el diccionario podría mostar sus dos primeros valores

```
print("Servidor:", servidor["nombre"])
print("IP:", servidor["ip"])
```

Y para el tercer valor se recorre la lista cuyo identificador es `servidor["servicios"]`

```
# print(servidor["servicios"])
print("Servicios:")
for servicio in servidor["servicios"]:
    print(" -", servicio)
```

Al ser una lista, para acceder a un element ose usa la notación de corchetes tras el nombre de la lista `servidor["servicios"][indice]`

```
# Imprime un servicio de la lista
print("Primer servicio:", servidor["servicios"][0])
```

Lectura de izquierda a derecha

Esta expresión:

```
servidor["servicios"][0]
```

puede interpretarse como:

1. busca la clave `"servicios"`;
2. obtiene la lista asociada;
3. accede al elemento situado en la posición `0`.

También puede utilizar los métodos habituales de las listas. En este caso se añade un servicio con `append`: No está creando una nueva clave sino modificando la lista:

```
servidor["servicios"].append("ftp")

print("Servicios tras añadir ftp:")
```

```
for servicio in servidor["servicios"]:
    print(" -", servicio)
```

2. Lista cuyos elementos son diccionarios

El segundo caso es muy habitual. Suponga que desea guardar información sobre varios servidores.

Puede crear una lista:

```
servidores = [
    {
        "nombre": "web01",
        "ip": "192.168.1.20"
    },
    {
        "nombre": "db01",
        "ip": "192.168.1.30"
    },
    {
        "nombre": "backup01",
        "ip": "192.168.1.40"
    }
]
```

En este caso tiene una **lista** con tres elementos. Y vada elemento es un **diccionario**.

Se puede acceder al primer elemento de la lista. Ese elemento es un diccionario.

```
print("Primer servidor:")
print(servidores[0])
```

El resultado será:

```
{
    "nombre": "web01",
    "ip": "192.168.1.20"
}
```

Al ser `servidores[0]` un diccionario se puede acceder a sus valores:

```
print("Nombre del primer servidor:")
print(servidores[0]["nombre"])
```

El resultado será:

```
web01
```

Y para recorrer la lista mostrando los valores

```
print("Lista de servidores:")
for servidor in servidores:
    print(servidor["nombre"], "=>", servidor["ip"])
```

3. Anexo. Recomendaciones

- Utilizar nombres de variables descriptivos. En estructuras combinadas es especialmente importante utilizar nombres claros. Por ejemplo:

  ```
  for servidor in servidores:
  ```

  es más fácil de entender que:

  ```
  for x in servidores:
  ```

- Separar los pasos POR CLARIDAD. En vez de:

  ```
  print(servidores[0]["nombre"])
  ```

  puede escribir:

  ```
  primer_servidor = servidores[0]
  print(primer_servidor["nombre"])
  ```

Ambas formas son correctas, pero la segunda puede resultar más legible al principio.

2.4 Ejercicios I

2.4.1 Ejercicios Tema 01

Ejercicio 1. Variables y tipos

Crea tres variables:

- `nombre`, con tu nombre.
- `edad`, con tu edad.
- `altura`, con tu altura en metros.

Muestra el contenido de las tres variables y, después, muestra el tipo de cada una utilizando `type()`.

Ejercicio 2. Cadenas

Partiendo de:

```
frase = "Python es divertido"
```

haz que el programa:

1. Muestre la frase en minúsculas.
2. Muestre su longitud.
3. Compruebe si contiene la palabra `"python"` sin distinguir mayúsculas y minúsculas.
4. Muestre la frase al revés.

Ejercicio 3. Booleanos

Crea una variable:

```
es_administrador = False
```

Utiliza un `if` y un `else` para mostrar:

```
Acceso de administrador
```

si su valor es verdadero, y:

```
Acceso de usuario
```

si es falso.

Después cambia su valor a `True` y observa el resultado.

Ejercicio 4. Conversión de tipos

Pide al usuario el precio de un producto y una cantidad:

```
Precio:
Cantidad:
```

Convierte los valores a los tipos adecuados y calcula el importe total:

```
importe = precio * cantidad
```

Muestra finalmente el resultado.

Ejercicio 5. Pequeño reto

Pide al usuario su nombre y su año de nacimiento. Calcula aproximadamente su edad suponiendo que estamos en 2026 y muestra un mensaje similar a:

```
Ana, tienes aproximadamente 20 años.
```

Pista

Recuerda que `input()` devuelve siempre una cadena. Para realizar la resta tendrás que convertir el año de nacimiento a un tipo numérico.

2.4.2 Ejercicios

Ejercicio 1. Mayor de edad

Pide al usuario su edad.

Si tiene 18 años o más, muestra:

```
Es mayor de edad
```

Si no, muestra:

```
Es menor de edad
```

Ejercicio 2. Número positivo, negativo o cero

Pide al usuario un número entero y determina si es:

- positivo;
- negativo;
- cero.

Utiliza `if`, `elif` y `else`.

Ejercicio 3. Acceso al sistema

Pide al usuario:

- su edad;
- si dispone de saldo.

Para simplificar, representa el saldo mediante un número.

Permite el acceso solamente si tiene al menos 18 años **y** el saldo es mayor que 0.

Ejercicio 4. Comparar dos números

Pide al usuario dos números enteros y muestra cuál es mayor.

Si son iguales, indícalo también.

Ejercicio 5. Menú con `match`

Muestra este menú:

```
1. Crear usuario
2. Eliminar usuario
3. Salir
```

Pide una opción y utiliza `match` para mostrar la acción seleccionada.

Para cualquier otro número, muestra:

```
Opción incorrecta
```

2.4.3 Ejercicios

Ejercicio 1. Contar del 0 al 9

Utiliza un bucle `for` y `range()` para mostrar los números del 0 al 9.

Ejercicio 2. Contar del 1 al 10

Modifica el ejercicio anterior para mostrar:

```
1
2
3
...
10
```

Ejercicio 3. Sumar del 1 a un número

Pide al usuario un número entero positivo.

Utiliza un acumulador para calcular la suma desde `1` hasta ese número.

Por ejemplo, si introduce `5`:

```
1 + 2 + 3 + 4 + 5 = 15
```

Ejercicio 4. Repetir hasta introducir cero

Pide números al usuario continuamente.

Cada vez que introduzca un número distinto de `0`, muéstralo.

Cuando escriba `0`, termina el programa.

Resuélvelo utilizando `while` **sin utilizar** `break`.

Ejercicio 5. El mismo problema con `break`

Repite el ejercicio anterior, pero esta vez utiliza:

```
while True:
```

y una instrucción `break`.

Ejercicio 6. Recorrer una palabra

Pide al usuario una palabra y muestra cada carácter en una línea distinta.

Por ejemplo:

```
Palabra: casa
c
a
s
a
```

Ejercicio 7. Mostrar posiciones y caracteres

Pide una palabra al usuario y muestra la posición y el carácter correspondiente.

Por ejemplo:

```
Palabra: sol
0: s
1: o
2: l
```

2.4.4 Ejercicios

Ejercicio 1. Crear y consultar una lista

Crea una lista con los puertos `22`, `53`, `80`, `443` y `8080`. Muestra la lista completa, el primer elemento, el último y el número de elementos.

Ejercicio 2. Modificar una lista

Partiendo de:

```
usuarios = ["ana", "luis", "marta"]
```

cambia `"ana"` por `"alfredo"`, añade `"raquel"` al final e inserta `"carlos"` en la posición `1`.

Ejercicio 3. Buscar una IP

Crea:

```
ips = ["192.168.1.1", "10.0.0.1", "172.16.0.1"]
```

Pide al usuario una IP y muestra si está o no en la lista utilizando `in`.

Ejercicio 4. Posición de una IP

Amplía el ejercicio anterior para mostrar también la posición cuando exista, evitando que se produzca `ValueError`.

Ejercicio 5. Ordenar edades

Crea:

```
edades = [34, 18, 55, 21, 40]
```

Ordénala de menor a mayor, muéstrala, inviértela y vuelve a mostrarla.

Ejercicio 6. Usuarios separados por comas

Pide nombres separados por comas, conviértelos en una lista con `split()` y muestra cada usuario en una línea.

Ejercicio 7. Calcular la media

Pide varias notas separadas por `;`, conviértelas a números y calcula la media.

Ejercicio 8. Conversión mediante comprensión

Partiendo de:

```
numeros = ["10", "20", "30", "40"]
```

crea una lista de enteros mediante una comprensión de listas.

2.4.5 Ejercicios

Ejercicio 1. Crear un diccionario

Crea un diccionario llamado `servidor` con estas claves:

- `"nombre"`
- `"ip"`
- `"puerto"`

Asigna valores de ejemplo y muestra el diccionario completo.

Ejercicio 2. Consultar elementos

Partiendo de:

```
usuario = {
    "nombre": "Ana",
    "edad": 21,
    "activo": True
}
```

muestra por separado:

1. el nombre;
2. la edad;
3. si está activo.

Ejercicio 3. Modificar y añadir información

Partiendo del diccionario anterior:

1. cambia la edad a `22`;
2. añade una clave `"email"`;
3. muestra el diccionario completo.

Ejercicio 4. Buscar una clave

Crea:

```
equipo = {
    "hostname": "server01",
    "ip": "192.168.1.10",
    "so": "Linux"
}
```

Pide al usuario el nombre de una clave.

Comprueba si existe utilizando:

```
in
```

y muestra su valor si existe.

Pista

Puedes comprobar una clave así:

```
if clave in equipo:
```

Ejercicio 5. Recorrer clave y valor

Recorre este diccionario:

```
puertos = {
    "ssh": 22,
    "http": 80,
    "https": 443
}
```

y muestra:

```
ssh => 22
http => 80
https => 443
```

Ejercicio 6. Utilizar `get()`

Partiendo de:

```
persona = {
    "nombre": "Luis",
    "ciudad": "Cádiz"
}
```

consulta con `get()`:

- `"nombre"`;
- `"telefono"`.

Para `"telefono"`, utiliza como valor por defecto:

```
No disponible
```

2.5 Módulos/funciones

2.5.1 Introducción a Python: uso módulos y funciones

En administración de sistemas, las funciones son bloques de código reutilizables diseñados para automatizar tareas repetitivas. Las funciones pueden ser propias o que ya vengan en módulos de la biblioteca estándar o de terceros

Por ejemplo, el módulo `os` de la biblioteca estándar de Python permite interactuar directamente con el sistema operativo (archivos, directorios, procesos y rutas) de forma independiente del SO.

1. Usando un módulo

Para invocar (ejecutar) una función que pertenece a un módulo en Python, se sigue esta sintaxis:

1. **Importar el módulo:** Declarar la dependencia al inicio del script (`import modulo`).
2. **Notación de punto y paréntesis:** Conectar el nombre del módulo con la función (`modulo.funcion()`).
3. **Argumentos entre paréntesis `()` :** Puede no tener argumentos, o que sean obligatorios u opcionales, puede necesitar uno o varios, etc.

2. Función SIN parámetros

Una función sin parámetros no requiere datos de entrada para realizar su trabajo; obtiene la información directamente del entorno de ejecución. Es el caso de `os.getcwd()` :

- **Propósito:** Devuelve la ruta absoluta del directorio de trabajo actual (*Get Current Working Directory*).
- **Equivalente CLI:** Comando `pwd` en Linux/macOS o `cd` en Windows CMD.

```
import os

# Se invoca dejando los paréntesis vacíos
directorio_actual = os.getcwd()

print(f"El script se está ejecutando desde: {directorio_actual}")
```

3. Función CON parámetro obligatorio

Una función con parámetros requiere que le pases uno o más datos (argumentos) dentro de los paréntesis para poder operar. Es el caso de `os.mkdir(path)` :

- **Propósito:** Crea un nuevo directorio en el sistema de archivos en la ruta indicada.
- **Equivalente CLI:** Comando `mkdir <nombre_directorio>` .

```
import os

# Definimos el argumento: el nombre de la carpeta que queremos crear
carpeta_backup = "/var/backups/diarios"

# Invocación pasando el parámetro (obligatorio)
os.mkdir(carpeta_backup)
```

```
print(f"Directorio creado correctamente en: {carpeta_backup}")
```

> Nota para administradores: Si la carpeta ya existe o no se tienen permisos de escritura en la ruta destino, Python interrumpirá el script con una excepción. A diferencia de los scripts de Bash o las llamadas en C (donde las funciones devuelven un código de salida como 0 o 1), os.mkdir() gestiona los resultados mediante el flujo de excepciones:
>
> - Si la carpeta se crea correctamente: La función finaliza en silencio devolviendo None.
> - Si ocurre un error: No devuelve un código de error, sino que interrumpe la ejecución lanzando una excepción:
>   - FileExistsError: La carpeta ya existe en esa ruta.
>   - PermissionError: El usuario que ejecuta el script no tiene permisos de escritura.
>   - FileNotFoundError: La ruta intermedia especificada no existe (por ejemplo, intentar crear a/b/c cuando a o b no existen).

4. Función CON parámetro opcional

El método `os.listdir(path)` retorna:

- con parametro `path` el contenido del directorio indicado en `path`
- sin parámetro `path` el contenido del directorio actual

```
# Método con argumento opcional
files_dir_actual = os.listdir()
files_dir_backup = os.listdir(carpeta_backup)
print(files_dir_actual)
print(files_dir_backup)
```

5. Función con más de un parámetro

Un ejemplo con dos parámetros (obligatorios) es la función `os.rename(src, dst)` que sirve para renombrar o mover un archivo o directorio de una ubicación a otra. Es equivalente al comando `mv` en Linux:

1. Primer parámetro (`src` / *source*): La ruta del archivo o carpeta original que ya existe.
2. Segundo parámetro (`dst` / *destination*): La ruta o nombre final del archivo o carpeta.

```
import os

# 1. Definimos los datos que necesita la función
log_actual = "/var/log/servidor.log"
log_rotado = "/var/log/servidor.log.old"

# 2. Invocamos la función pasando ambos parámetros separados por una coma
#    El orden es importante
os.rename(log_actual, log_rotado)

print("El archivo se ha renombrado correctamente.")
```

> Al igual que `os.mkdir()`, devuelve `None` y gestiona los fallos mediante excepciones (`FileNotFoundError`, `PermissionError`, etc.).

6. Argumentos: Posicionales vs con nombre

En Python cuando una función recibe varios parámetros se pueden pasar de dos formas. Usemos como ejemplo `os.rename(src, dst)`:

1. Por posición (lo más habitual):El orden de los argumentos dentro del paréntesis determina a qué parámetro corresponde cada uno (el primer valor va a `src` y el segundo a `dst`).

```
os.rename("/tmp/archivo.txt", "/tmp/archivo_nuevo.txt")
```

2. Por nombre de parámetro (*keyword arguments*): Puede especificar explícitamente el nombre del parámetro. En este caso, el orden en el que los escriba no importa.

```
os.rename(dst="/tmp/archivo_nuevo.txt", src="/tmp/archivo.txt")
```

7. Ejemplo completo

```
import os

# Método sin argumentos:
# Se invoca dejando los paréntesis vacíos
directorio_actual = os.getcwd()
print(f"El script se está ejecutando desde: {directorio_actual}")

# Método con argumento obligatorio:
# Definimos el argumento: el nombre de la carpeta que queremos crear
carpeta_backup = "bk"

# Invocación pasando el parámetro (obligatorio)
os.mkdir(carpeta_backup)
print(f"Directorio creado correctamente en: {carpeta_backup}")

# Método con argumento opcional
files_dir_actual = os.listdir()
files_dir_backup = os.listdir(carpeta_backup)
print(files_dir_actual)
print(files_dir_backup)


## Método con 2 argumentos posicionales
# 1. Definimos los datos que necesita la función
log_actual = "servidor.log"
log_rotado = "servidor.log.old"

# 1.1 Crear archivo para que rename no falle,,,
with open(log_actual, mode="a", encoding="utf-8") as archivo:
    pass

# 2. Invocamos la función pasando ambos parámetros separados por una coma
os.rename(log_actual, log_rotado)

print("El archivo se ha renombrado correctamente.")


# Método con argumentos con nombre
os.rename(dst="/tmp/archivo_nuevo.txt", src="/tmp/archivo.txt")
```

2.5.2 Introducción a Python: Funciones II

Las funciones son bloques de código reutilizables diseñados para automatizar tareas repetitivas.

- Para definir una función en Python, se utiliza la palabra reservada `def`, seguida del nombre de la función, paréntesis `()` y dos puntos `:`. Todo el código que pertenezca a la función debe estar indentado (desplazado a la derecha).
- Para ejecutar una función hay que invocarla por su nombre con los paréntesis `()`.

1. Funciones SIN parámetros

Una función sin parámetros no requiere datos de entrada para realizar su trabajo.

```
# Función sin parámetros (siempre hace lo mismo)
def imprime_separador_simple():
    print("-"*25)

# Invocación ejecuta la función
imprime_separador_simple()
imprime_separador_simple()
```

2. Funciones CON un parámetro

Una función con parámetro requiere que le pase un parámetro o argumento dentro de los paréntesis para poder operar de forma dinámica.

```
# Función con un parámetro
def imprime_separador(longitud):
    print("-"*longitud)

# Invocaciones con diferentes expresiones
imprime_separador(5)
imprime_separador(10)
tam = 15
imprime_separador(tam)
imprime_separador(tam*2)
```

3. Funciones CON múltiples parámetros

Puedes definir tantos parámetros como necesites, separándolos por comas. Cuando una función recibe varios parámetros, los datos se pueden enviar en la invocación de dos formas distintas:

```
# Función con dos parámetros
def imprime_linea(letra, longitud):
    print(letra*longitud)

# Invocaciones posicionales con diferentes valores
imprime_linea("*", 5)
imprime_linea("#", tam)

# Invocación con nombres, puede cambiar el orden
imprime_linea(longitud=10, letra="&")
```

4. Función que usa otra funcion

```
# Utiliza en la definición otra función
def imprime_cabecera(titulo):
    imprime_linea("+",10)
    print(titulo)
    imprime_linea("-",10)

imprime_cabecera("Latencias")
```

5. Funciones que retornan un resultado (return)

En Python las funciones por defecto no devuelven ningún valor explícito, retornando `None`. Sin embargo, si necesita que la función devuelva un cálculo final al script principal se utiliza la palabra clave `return`.

```
# función que DEVUELVE COMO RESULTADO la suma de valores de una lista
def sumaLista(lista):
```

```
    suma = 0
    for x in lista:
        suma += x   # suma = suma + x
    return suma

ventas = [1,2,3,4,5]
print("suma: ", sumaLista(ventas))

ventasTotales = sumaLista(ventas)
print("ventas:", ventas)
print("ventasTotales:", ventasTotales)
```

```
# función que DEVUELVE COMO RESULTADO la media USANDO LA FUNCIÓN sumaLista
def media(lista):
    media = sumaLista(lista)/len(lista)
    return media

edades = [23, 46, 56, 72, 12, 34]
print("edades:", edades)
print("sumaLista(edades):", sumaLista(edades))
print("media(edades):", media(edades))
```

Valores por defecto

Se pueden definir valores por defecto para los argumentos

```
# ejemplo valores por defecto
def imprime_test(longitud=20, letra="-"):
    print(letra*longitud)

imprime_test()
imprime_test(10)
imprime_test(15, "*")
```

Tenga en cuenta que en las llamadas a las funciones el orden de los parámetros debe coincidir con el orden de los argumentos en la función:

```
# Parametro posicionales
def carga_recibo(saldo, recibo):
    if recibo < 0:
        print("recibo debe ser > 0")
    elif saldo < recibo:
        print("saldo inferior a recibo")
    else:
        saldo = saldo - recibo

    return saldo
```

Y un ejemplo de ejecución correcta e incorrecta:

```
saldo_actual = 100
recibo_luz = 45
recibo_agua = 40
print("Saldo: ", saldo_actual)

# El orden importa, sin nombres la correspondencia es posicional
saldo_actual = carga_recibo(saldo_actual, recibo_luz)  # Correcto
print("Saldo tras recibo: ", saldo_actual)
print("llamada incorrecta que además modifica el saldo !!!!")
saldo_actual = carga_recibo(recibo_agua, saldo_actual)  # incorrecto !!!!
print("Saldo tras recibo: ", saldo_actual)
```

Y un ejemplo con uso de nombres:

```
# otra opción: con nombres
saldo_actual = carga_recibo(recibo=recibo_luz, saldo=saldo_actual) # con nombres
```

6. Anexo. Uso de Módulos

Otro concepto interesantes es el de los módulos en Python:

> Modules are files that contain Python code, which allow us to divide code into multiple files so that it is more organized and easier to reuse code.
>
> A module in Python is simply a file with a .py extension that contains Python code (within this file, we can include function definitions, variables, classes, etc.).

Tiene una explicación detallada con ejemplos en este enlace: https://www.luisllamas.es/en/what-are-python-modules/

Un caso práctico a partir de los ejemplos previos: se agrupan las funciones `sumaLista`, `media`, etc, en un fichero llamado `utiles.py`.

utiles.py

```
# DEVUELVE COMO RESULTADO la suma de valores de una lista
def sumaLista(lista):
    suma = 0
    for x in lista:
        suma += x   # suma = suma + x
    return suma

# DEVUELVE COMO RESULTADO la media USANDO LA FUNCIÓN sumaLista
def media(lista):
    media = sumaLista(lista)/len(lista)
    return media
```

Y otro fichero con la aplicación:

main.py

```
# El nombre del módulo a importar es el nombre del fichero
# Se importan las funciones que use del móudlo
from utiles import sumaLista, media

ventas = [1,2,3,4,5]
ventasTotales = sumaLista(ventas)

print("Lista ventas:", ventas)
print("Suma Ventas:", ventasTotales)
```

Y la ejecuión y su salida

```
$ python3 main.py
Lista ventas: [1, 2, 3, 4, 5]
Suma Ventas: 15
$
```

2.6 Introducción a Python - Excepciones

2.6.1 1. Introducción

Cuando un programa encuentra un problema durante su ejecución, Python puede generar una **excepción**. Ya vimos un ejemplo al utilizar `os.mkdir()` para crear directorios. Por ejemplo:

```
import os

os.mkdir("/root/nueva_carpeta")
```

Si el usuario que ejecuta el programa no tiene permisos para crear el directorio, Python interrumpe la ejecución y genera una excepción. Una posible salida sería:

```
Traceback (most recent call last):
  File "13_excepciones.py", line 25, in <module>
    os.mkdir("/root/nueva_carpeta")
PermissionError: [Errno 13] Permission denied: '/root/nueva_carpeta'
```

En este caso, la excepción es:

```
PermissionError
```

2.6.2 2. ¿Qué es una excepción?

Una excepción representa una situación anómala que ocurre mientras el programa se está ejecutando. Algunos ejemplos habituales son:

- intentar convertir a número un texto que no contiene un número;
- dividir entre cero;
- acceder a una posición que no existe en una lista;
- buscar una clave inexistente en un diccionario;
- abrir un fichero que no existe;
- crear un directorio sin permisos suficientes.

Si no tratamos la excepción, el programa normalmente se detiene y Python muestra un **traceback**.

> **Excepción no significa necesariamente error de programación**
>
> Una excepción también puede representar situaciones perfectamente posibles durante la ejecución.
>
> Por ejemplo, que un usuario escriba un dato incorrecto o que un fichero no exista.

2.6.3 3. Excepciones con `os.mkdir()`

La función `os.mkdir(ruta)` no devuelve un código de error cuando algo falla. Si el directorio se crea correctamente, la función termina y devuelve:

```
None
```

Si se produce un problema, genera una excepción. Algunas excepciones habituales son:

- `FileExistsError`: Se produce si el directorio ya existe.
- `PermissionError`: Se produce si no tenemos permisos suficientes.
- `FileNotFoundError`: Puede aparecer si algún directorio intermedio no existe.

2.6.4 4. La estructura `try ... except`

Para evitar que el programa se detenga automáticamente podemos **capturar** la excepción.La estructura básica es:

```
try:
    # código que puede generar una excepción
except TipoDeExcepcion:
    # qué hacer si ocurre
```

Por ejemplo:

```
import os

try:
    os.mkdir("/root/nueva_carpeta")
except PermissionError:
    print("No tiene permisos para crear el directorio")
```

2.6.5 5. Tratar varias excepciones

Podemos utilizar varios bloques `except` .

```
import os

ruta = input("Directorio a crear: ")

try:
    os.mkdir(ruta)

except FileExistsError:
    print("El directorio ya existe")

except PermissionError:
    print("No tiene permisos para crear el directorio")

except FileNotFoundError:
    print("Algún directorio intermedio no existe")
```

Cada `except` trata un tipo de excepción diferente.

2.6.6 6. Obtener información sobre la excepción

Podemos guardar el objeto de la excepción en una variable. Vea un ejemplo:

```
try:
    os.mkdir("/root/nueva_carpeta")

except PermissionError as e:
    print("Error de permisos")
    print("Detalle:", e)
```

La variable `e` contiene información generada por Python sobre la excepción.

2.6.7 7. Captura genérica con `Exception`

También podemos capturar la excepción sin especificar el tipo usando `Exception` . Sería similar a un "caso por defecto";

```
try:
    os.mkdir(ruta)

except FileExistsError:
    print("El directorio ya existe")

except PermissionError:
    print("No tiene permisos")

except Exception as e:
    print("Se ha producido otro error:", e)
```

> ⚠ **Las excepciones específicas deben ir primero**
>
> Los casos específicos deben aparecer antes que `Exception` , dejando el caso genérico al final.

Usar sólo una excepción genérica puede ocultar información útil. Es preferible capturar excepciones concretas cuando sabemos cuáles pueden ocurrir y utilizar `Exception` solo como último caso cuando tenga sentido.

2.6.8 8. Ejemplos excepciones

8.1. `input()` y `ValueError`

```
try:
    edad = int(input("Edad: "))
    print("Edad:", edad)

except ValueError:
    print("Debe introducir un número entero")
```

Si el usuario escribe un texto que no se puede convertir con `int()`, se produce `ValueError`.

8.2. división por cero

```
try:
    a = int(input("Dividendo: "))
    b = int(input("Divisor: "))

    resultado = a / b
    print("Resultado:", resultado)

except ValueError:
    print("Debe introducir números enteros")

except ZeroDivisionError:
    print("No se puede dividir entre cero")
```

Aquí pueden ocurrir dos problemas distintos:

- el usuario introduce algo que no es un entero;
- el divisor es `0`.

2.6.9 9. Anexo I: `else` y `finally`

9.1. else

Se puede añadir un bloque `else` a `try` qeu se ejecuta **solo si no se produce ninguna excepción**.

```
try:
    numero = int(input("Número: "))

except ValueError:
    print("Valor incorrecto")

else:
    print("Número correcto:", numero)
```

9.2. `finally`

El bloque `finally` se ejecuta **siempre**, tanto si se produce una excepción como si no.

```
try:
    numero = int(input("Número: "))
    print("Número:", numero)

except ValueError:
    print("Valor incorrecto")

finally:
    print("Fin del intento")
```

`finally` es útil para tareas de limpieza que deben ejecutarse siempre.

9.3. Estructura completa

```
try:
    ...
```

```
except TipoError:
    ...
else:
    ...
finally:
    ...
```

El significado es:

- `try` : código que puede generar una excepción;
- `except` : qué hacer si ocurre;
- `else` : qué hacer si todo ha ido bien;
- `finally` : qué hacer siempre.

No es necesario utilizar todos los bloques.

2.6.10 10. Ejemplo completo con directorios

```
import os

ruta = input("Directorio a crear: ")

try:
    os.mkdir(ruta)

except FileExistsError:
    print("El directorio ya existe")

except PermissionError:
    print("No tiene permisos para crear el directorio")

except FileNotFoundError:
    print("Algún directorio intermedio no existe")

except Exception as e:
    print("Se ha producido un error no previsto:")
    print(e)

else:
    print("Directorio creado correctamente")

finally:
    print("Fin del intento de creación")
```

2.6.11 11. Anexo II: Excepciones habituales en Python

Excepción	Ejemplo
`ValueError`	`int("hola")`
`ZeroDivisionError`	`10 / 0`
`IndexError`	acceder fuera de una lista
`KeyError`	usar una clave inexistente
`FileNotFoundError`	abrir un fichero inexistente
`FileExistsError`	crear un directorio que ya existe
`PermissionError`	acceder sin permisos

11.1. `IndexError` con listas

```
puertos = [22, 80, 443]

try:
    posicion = int(input("Posición: "))
    print(puertos[posicion])

except ValueError:
    print("Debe introducir un número")
```

```
except IndexError:
    print("La posición no existe")
```

11.2. `KeyError` con diccionarios

```
servidor = {
    "nombre": "web01",
    "ip": "192.168.1.20"
}

try:
    print(servidor["puerto"])

except KeyError:
    print("La clave no existe")
```

Para una consulta sencilla también podemos utilizar:

```
servidor.get("puerto")
```

o comprobar previamente:

```
if "puerto" in servidor:
```

> **No todo debe resolverse con `try`**
>
> A veces existe una forma más sencilla de evitar el problema.

2.6.12 12. Anexo III: Recomendaciones

12.1. Evitar bloques `try` demasiado grandes

Es preferible limitar el `try` al código que realmente puede generar la excepción que queremos controlar.

Por ejemplo:

```
try:
    edad = int(input("Edad: "))
except ValueError:
    print("Edad incorrecta")
```

resulta más claro que incluir muchas operaciones no relacionadas dentro del mismo `try`.

12.2. Evitar `except:` sin indicar excepción

Python permite:

```
except:
    print("Error")
```

pero normalmente conviene evitar esta forma. Es preferible:

```
except ValueError:
```

o si necesitamos un caso genérico:

```
except Exception as e:
```

2.7 Ejercicios II

2.7.1 Ejercicios

Ejercicio 1. Directorio actual

Utiliza el módulo `os` para mostrar el directorio actual.

Ejercicio 2. Contenido de `/tmp`

Obtén el contenido de `/tmp` con `os.listdir()` y listas el contenido con un bucle `for`.

Ejercicio 3. Crear directorio

Pide al usuario un nombre y crea dentro de `/tmp` un directorio con ese nombre.

Ejercicio 4. Renombrar un archivo

Suponga que existe `/tmp/informe.txt`. Utiliza `os.rename()` para renombrarlo como `/tmp/informe.old`.

2.7.2 Ejercicios

Ejercicio 1. Información de un servidor

Crear una función:

```
muestra_servidor(servidor)
```

que recibe un diccionario con claves `nombre` e `ip` de un equipo y los muestra por pantalla, cada uno en una línea.

Ejercicio 2. Parámetro opcional

Modificar el ejercicio anterior para añadir al diccioanrio la clave `activo` de tipo lógico. La función se modifica para mostrar en su salida la cadena "Up" o "Down" en función de dicha clave.

Ejercicio 3. Validar un puerto

Crear la función:

```
puerto_valido(puerto)
```

que devuelva `True` si el puerto está entre `1` y `65535` y `False` en caso contrario.

Más ejercicios

1. Función que devuelve `True` si un número entero está entre 1 y 10, `False` en caso contrario.
2. Función que solicita un número entero al usuario. Si el valor no está entre 1 y 10 solicita de nuevo el número. Devuelve como resultado de la función el número.
3. Función que recibe como argumento `login` y `password`. Devuelve `True` si la clave es correcta o `False` si no (si la longitud es menor que 10, o el `login` incluido en el `password`)

Y otra tanda

1. Solicitar una IP por teclado. Comprobar que no esté incluida en la lista `bloqueados = ["10.10.10.100", "192.168.1.200", "8,8,8,8"]`.
2. Solicitar por teclado un string con números de puertos separados por espacios. Crear a partir de esa entrada una lista de puertos.
3. Dado el diccionario `servicios = {"ssh": 22, "http":80, "ftp": 21}` escriba una función que use un bucle for para imprimir solo los nombres de los servicios (las claves).
4. Usando el mismo diccionario crear función que use el metodo `.items()` para imprimir para cada servicio `El servicio xxx usa el puerto yyy`.

2.7.3 Ejercicios

1. Ejercicio 1. Conversión a entero

Pide al usuario un número entero.

Si introduce un valor que no pueda convertirse con `int()`, muestra:

```
Debe introducir un número entero
```

2. Ejercicio 2. División

Pide dos números enteros y divide el primero entre el segundo.

Controla:

- `ValueError`;
- `ZeroDivisionError`.

3. Ejercicio 3. Crear un directorio

Pide al usuario el nombre de un directorio y créalo con:

```
os.mkdir()
```

Controla al menos:

- `FileExistsError`;
- `PermissionError`.

Si se crea correctamente, muestra:

```
Directorio creado
```

4. Ejercicio 4. Acceso a una lista

Partiendo de:

```
servicios = ["ssh", "http", "https"]
```

pide una posición y muestra el servicio correspondiente.

Controla:

- `ValueError`;
- `IndexError`.

5. Ejercicio 5. Diccionario de servidor

Partiendo de:

```
servidor = {
    "nombre": "web01",
    "ip": "192.168.1.20",
    "puerto": 80
}
```

pide una clave al usuario e intenta mostrar su valor utilizando:

```
servidor[clave]
```

Controla `KeyError`.

2.8 Prácticas

2.8.1 Práctica Bloque 01

Fichero con datos: servidores.py

1. Apartados

- Apartado 01
- Apartado 02
- Apartado 03
- Apartado 04
- Apartado 05
- Apartado 06
- Apartado 07
- Apartado 08

2.8.2 Práctica: Control de catálogo

1. Situación de partida

Realizar una herramienta de consola para **consultar y supervisar un catálogo de productos**. La información no estará escrita dentro de tu programa: se obtendrá de una API REST pública llamada **DummyJSON**. A partir de los datos recuperados desarrollarás, paso a paso, un programa para listar productos, buscarlos, filtrarlos, calcular estadísticas, etc. La práctica repasa estructuras de control, listas, diccionarios, funciones y excepciones.

> **novedades**
>
> Se introducen únicamente dos novedades: la petición HTTP con `requests` y la conversión de JSON a estructuras Python mediante `.json()`.

1.1. OBJETIVOS DE APRENDIZAJE

Al terminar deberías poder:

- Recorrer y transformar **listas de diccionarios**.
- Acceder a **diccionarios y listas anidados**.
- Diseñar funciones que reciben y devuelven estructuras de datos.
- Validar datos y controlar **excepciones**.

2. Preparación

2.1. REQUISITO BIBLIOTECA `requests`

Es posible que el módulo `requests` está ya instalado. Lo puede comprobar con esta secuencia:

```
$ python3
Python 3.13.5 (main, Aug 10 2026, 12:06:59) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import requests
>>> quit()
$
```

Si no desde terminal, preferiblemente en un entorno virtual:

```
python3 -m venv venv
source venv/bin/activate
pip install requests
```

2.2. USO API DUMMYJSON

Comprueba primero en el navegador o con `curl` que esta dirección devuelve un documento JSON:

https://dummyjson.com/products?limit=3

El **diccionario** tiene una estructura parecida a esta (ejemplo abreviado):

```
{
  "products": [
    {
      "id": 1,
      "title": "Producto de ejemplo",
      "category": "beauty",
      "price": 9.99,
      "stock": 5,
      "rating": 4.4,
      "tags": ["beauty", "mascara"],
      "dimensions": {"width": 23.17, "height": 14.43, "depth": 28.01},
      "reviews": [
        {"rating": 5, "reviewerName": "Cliente A", "comment": "Muy bien"},
        {"rating": 2, "reviewerName": "Cliente B", "comment": "Mejorable"}
      ]
    }
  ],
  "total": 194,
  "skip": 0,
```

```
  "limit": 30
}
```

Observe que:

- El diccionario tiene 4 campos, el primero es `products`
- `products` es una **lista** de productos.
- Cada producto es un **diccionario**.
- Un producto contine entre otros
    - una lista: `tags`
    - un diccionario: `dimensions`
    - una lista de diccionarios: `reviews`
    - etc

Si lo desea puede consultar la documentación oficial de productos.

3. Apartados

3.1. APARTADO 1. CONEXIÓN CON LA API

Se le proporciona esta función ya terminada, y otra que simula la consulta por si no funciona la API o no tiene acceso a internet.

def obtener_productos

```python
def obtener_productos(limite=10):
    """Consulta DummyJSON y devuelve la lista 'products'.

    Propaga RequestException si falla HTTP o la conexión y ValueError si
    la respuesta no contiene la estructura esperada.
    """
    if not isinstance(limite, int) or isinstance(limite, bool) or limite <= 0:
        raise ValueError("El límite debe ser un entero positivo")

    respuesta = requests.get(
        URL_PRODUCTOS,
        params={"limit": limite},
        timeout=10,
    )
    respuesta.raise_for_status() #  Si servidor devuelve error lanza excepción HTTPError
    datos = respuesta.json()

    if not isinstance(datos, dict) or not isinstance(datos.get("products"), list):
        raise ValueError("Respuesta inesperada: falta la lista 'products'")

    return datos["products"]
```

En ela puede observar que

1. Realiza una petición `GET` a `https://dummyjson.com/products` utilizando `requests.get()`; envía `limite` como parámetro `limit`.
2. Establece un tiempo de espera (`timeout`) de 10 segundos.
3. Comprueba si la respuesta HTTP indica un error mediante `raise_for_status()`.
4. Convierte la respuesta JSON en estructuras Python mediante `.json()`.
5. Comprueba que la respuesta sea un diccionario que contenga la clave `products` y que su valor sea una lista. Si no lo es, lanza `ValueError` con un mensaje claro.
6. Devuelve **solo la lista** de productos.

3.2. APARTADO 2. LISTAR PRODUCTOS

Implementa:

```python
def mostrar_productos(productos):
    ...
```

Muestra, para cada producto, su `id`, `title`, `category`, `price` y `stock` en una tabla de texto legible. Si recibes una lista vacía, muestra `No hay productos que mostrar.`

Ejemplo de formato (valores ficticios):

```
 ID  PRODUCTO                      CATEGORÍA            PRECIO     STOCK
  1  Teclado USB                   accessories           25.90         8
  2  Monitor 24 pulgadas           electronics          159.00         3
```

Puedes limitar visualmente la anchura del título sin cambiar su valor real en el diccionario.

??? tip "Pista: presentación" Utiliza un `for` y cadenas con formato (`f"{valor:<20}"` , `f"{precio:>8.2f}"`). La función muestra datos; no necesita devolver ninguna lista nueva.

3.3. APARTADO 3. BUSCAR Y FILTRAR

Implementa dos funciones que **devuelvan nuevas listas de referencias a productos**; no imprimirán nada y no modificarán el catálogo:

```python
def buscar_por_titulo(productos, texto):
    ...

def filtrar_productos(productos, categoria=None, precio_max=None, solo_disponibles=False):
    ...
```

Búsqueda: `buscar_por_titulo` devuelve productos cuyo título contiene el texto indicado, sin distinguir mayúsculas de minúsculas. Una cadena vacía puede devolver todos los productos.

Filtros combinables: `filtrar_productos` permite seleccionar por categoría (coincidencia exacta, sin distinguir mayúsculas/minúsculas), precio máximo (incluido en el rango) y disponibilidad (`stock > 0`).

Todos los filtros activos se combinan mediante una condición lógica **Y**.

Ejemplo de uso:

```python
resultado = buscar_por_titulo(productos, "phone")
mostrar_productos(resultado)

resultado = filtrar_productos(
    productos,
    categoria="beauty",
    precio_max=30,
    solo_disponibles=True
)
mostrar_productos(resultado)
```

No presupongas cuántos resultados habrá: los productos publicados en la API pueden cambiar.

??? tip "Pista: comparación de cadenas" Puedes utilizar `.lower()` (o `.casefold()`) y `.strip()` para normalizar texto. Construye una lista vacía, recorre los productos y añade con `.append()` los que cumplan todos los criterios aplicables.

3.4. APARTADO 4. ESTADÍSTICAS

Implementa:

```python
def calcular_estadisticas(productos):
    ...
```

Debe **devolver un diccionario** con estas claves:

Clave	Contenido
`total`	Cantidad de productos analizados
`precio_medio`	Media aritmética de precios, redondeada a 2 decimales
`mas_caro`	Diccionario del producto de precio más alto (o `None` si no hay productos)
`unidades_en_stock`	Suma de las existencias de todos los productos
`por_categoria`	Diccionario `categoría: número de productos`

La función debe funcionar con cualquier lista, **también vacía**. En ese caso devuelve cero para las cantidades y la media, `None` para `mas_caro` y `{}` para `por_categoria` .

Después crea una pequeña función de presentación o muestra el resultado desde el menú. No introduzcas llamadas a la API en esta función.

??? tip "Pista: contar por categorías" Piensa en un diccionario que se inicializa vacío. Cada vez que aparece una categoría, aumenta su contador. Evita dividir entre cero cuando no hay productos.

4. Datos para cálculos independientes Internet

Puedes copiar temporalmente esta lista dentro de tu script y llamar directamente a tus funciones para comparar resultados

```
PRODUCTOS_EJEMPLO = [
    {
        "id": 1, "title": "Router", "category": "redes",
        "price": 80.0, "stock": 2, "rating": 2.5,
        "tags": ["wifi", "redes"],
        "dimensions": {"width": 20, "height": 5, "depth": 15},
        "reviews": [{"rating": 5}, {"rating": 3}, {"rating": 4}],
    },
    {
        "id": 2, "title": "Switch", "category": "redes",
        "price": 120.0, "stock": 0, "rating": 4.1,
        "tags": [], "dimensions": {}, "reviews": [],
    },
    {
        "id": 3, "title": "UPS", "category": "energia",
        "price": 100.0, "stock": 9, "rating": 4.5,
        "tags": ["rack"], "dimensions": {},
        "reviews": [{"rating": 2}, {"rating": 5}],
    },
]
```

5. Recursos

- DummyJSON — Documentación de productos
- Documentación de `requests`
- Tutorial oficial de estructuras de datos en Python

3. Tools

3.1 Introducción a Python - Linters

3.1.1 1. Introducción

Hay determinados problemas en el código que pueden detectarse **antes de ejecutar el programa**:

- errores de sintaxis;
- nombres que no existen;
- módulos importados pero no utilizados;
- variables creadas pero no usadas;
- código poco legible o difícil de mantener;
- uso de malas prácticas;
- estilos de código inconsistentes.

Las herramientas que analizan el código buscando este tipo de problemas se denominan *linters*. En Python existen distintas herramientas, entre ellas:

- Flake8
- Pylint
- Pyflakes
- Ruff
- ...

En este tema utilizaremos principalmente **Flake8**. Flake8 reúne comprobaciones procedentes de herramientas como:

- **Pyflakes**, para detectar determinados errores en el código;
- **pycodestyle**, para comprobar convenciones de estilo;
- **McCabe**, para medir complejidad ciclomática.

Entre otras cosas puede detectar imports no utilizados, nombres no definidos, problemas de espaciado, líneas demasiado largas y funciones excesivamente complejas.

> **¿Qué hace un linter?**
>
> Un linter realiza **análisis estático** del código: examina el código fuente sin necesidad de ejecutarlo.
>
> Un linter puede detectar muchos problemas, pero no puede garantizar que el programa funcione correctamente. Para comprobar el comportamiento del programa necesitamos técnicas como el **testing**.

1.1. Linter, formatter y testing

Conviene distinguir tres tipos de herramientas:

Herramienta	Objetivo
Linter	Detectar problemas, malas prácticas y errores de estilo
Formatter	Modificar automáticamente el formato del código
Testing	Comprobar que el programa produce los resultados esperados

Por ejemplo:

- `flake8` es principalmente un **linter**;
- `black` es un **formatter**;
- `pytest` permite realizar **tests**.

Estas herramientas se complementan entre sí.

A continuación:

1. se aprte de un fichero con errores: `test_linters_inicial.py`;
2. se crea un entorno virtual;
3. y se instala `flake8`;
4. se prueba el linter desde la línea de comandos;
5. y se utiliza integrado con el editor Visual Studio Code;

3.1.2 2. Instalación

2.1. Entorno virtual

- Crear y activar

```
calidad$ python -m venv venv
calidad$ source venv/bin/activate
(venv) calidad$
```

- Instalar `flake8`

```
(venv) @tos:~/calidad$ pip install flake8
```

3.1.3 3. Ejecución en CLI

Se el indica a `flake8` el fichero que desea revisar:

```
(venv) calidad$ flake8 test_linters_inicial.py
test_linters_inicial.py:8:1: F401 'io' imported but unused
test_linters_inicial.py:10:1: F403 'from math import *' used; unable to detect undefined names
test_linters_inicial.py:14:1: E303 too many blank lines (3)
test_linters_inicial.py:28:5: F841 local variable 'some_global_var' is assigned to but never used
test_linters_inicial.py:30:15: E225 missing whitespace around operator
test_linters_inicial.py:41:80: E501 line too long (80 > 79 characters)
test_linters_inicial.py:51:10: E711 comparison to None should be 'if cond is not None:'
test_linters_inicial.py:55:25: E703 statement ends with a semicolon
test_linters_inicial.py:63:23: E275 missing whitespace after keyword
test_linters_inicial.py:63:24: E201 whitespace after '('
test_linters_inicial.py:66:13: E303 too many blank lines (2)
test_linters_inicial.py:72:58: E251 unexpected spaces around keyword / parameter equals
test_linters_inicial.py:74:28: E221 multiple spaces before operator
...
(venv) calidad$
```

Observe como cada mensaje tiene un foramto similar `fichero:línea:columna: código descripción`

- el **fichero**,
- **línea** y columna
- el **código del error**
- y una **breve descripción** de ese código de error

Por ejemplo, en la línea 28 detecta el error F481: a la variable `some_global_var` se le asigna un valor pero esa variable no es usada posteriormente.

El programa puede seguir funcionando

Muchos avisos de un linter no provocan necesariamente un error durante la ejecución.

Una variable no utilizada, por ejemplo, no impide que el programa funcione, pero puede indicar código innecesario o un olvido del programador.

Hay también avisos de *estilo*: líneas de más de 79 caracteres, demasiados espacios después de un operador, etc.

Los códigos de error se pueden consultar en la documentación oficial. Y en este enlace tiene ejemplos descriptivos de los mismos.

3.1.4 4. Algunos códigos habituales

No corregir sin pensar

Un linter **señala problemas**, pero normalmente es el programador quien decide cómo corregirlos.El objetivo no es conseguir que desaparezcan todos los mensajes a cualquier precio.

Hay que comprender qué está señalando cada aviso y decidir cuál es la corrección adecuada.

- F401 - Import no utilizado

```
import os
```

Si posteriormente no se utiliza `os` muestra `F401 'os' imported but unused`

- F841 - Variable local asignada pero no utilizada

```
def ejemplo():
    resultado = 10
    print("Fin")
```

Si `resultado` nunca se utiliza muestra `F841 local variable 'resultado' is assigned to but never used` ```

- E225 - Falta espacio alrededor de un operador

No recomendado:

```
total=precio*cantidad
```

Recomendado:

```
total = precio * cantidad
```

- E501 - Línea demasiado larga

Podemos dividir expresiones largas utilizando paréntesis:

```
mensaje = (
    "Este texto se ha dividido "
    "en varias líneas del código."
)
```

Analizar un directorio completo

Puede analizar el directorio actual:

```
flake8 .
```

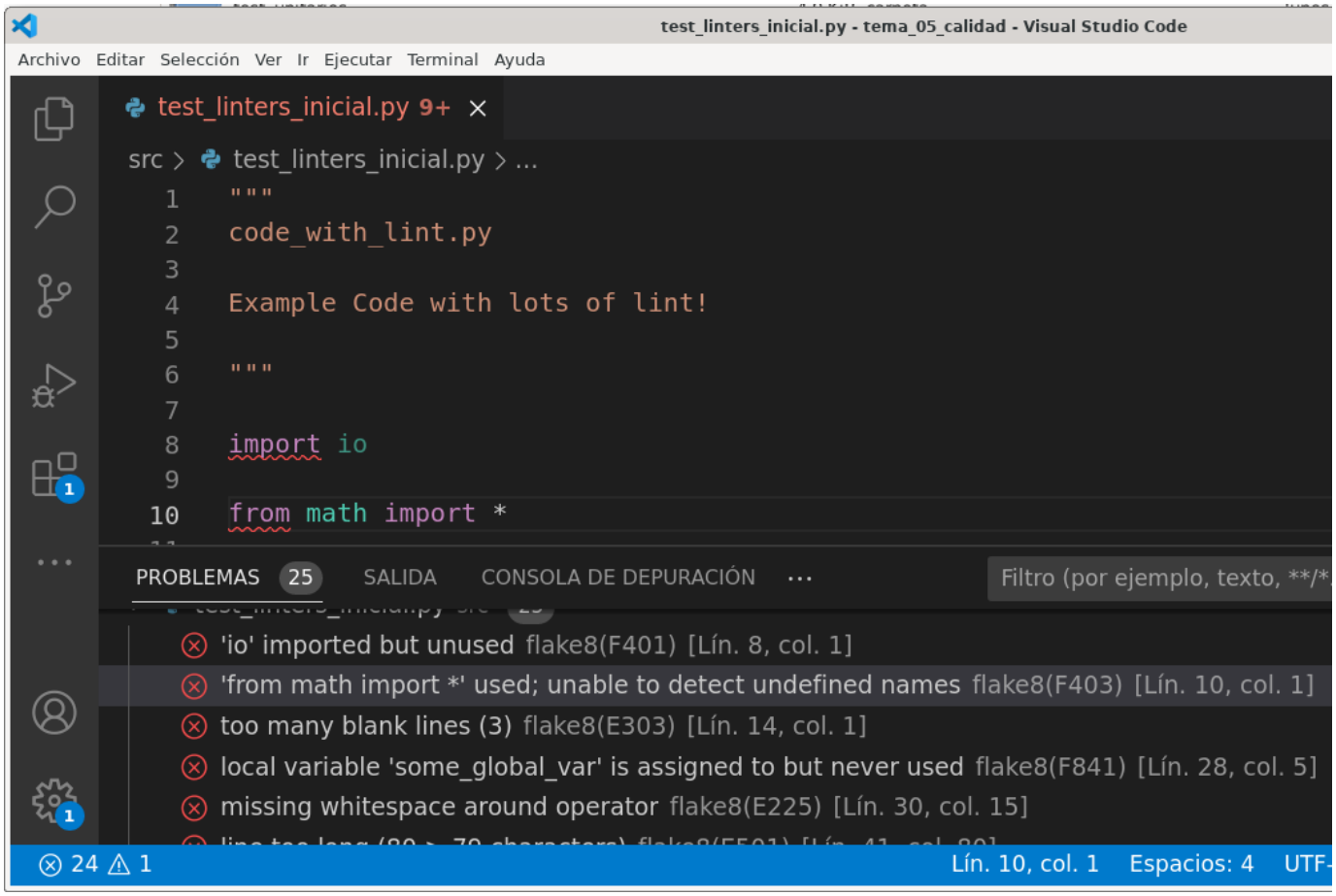
O una carpeta concreta:

```
flake8 src/
```

3.1.5 5. Usando el linter en el editor

Visual Studio Code dispone actualmente de una extensión **Flake8** publicada por Microsoft. Una vez instalada:

- los problemas aparecen directamente en el editor;
- también aparecen en el panel **Problemas**;
- el análisis puede ejecutarse automáticamente al trabajar con archivos Python.



Configuración antigua de VS Code

En versiones antiguas se elegía el linter mediante `Python: Select Linter` que debái estar previamente instalado.

Ese procedimiento está obsoleto. La extensión actual incluye su propia versión de Flake8, aunque también puede configurarse para usar una instalación del entorno del proyecto.

Para este curso es suficiente recordar:

1. instalar Flake8 en el entorno virtual para usarlo desde terminal;
2. instalar la extensión **Flake8** de Microsoft para usarlo desde VS Code.

3.1.6 6. Seleccionar reglas

Podemos revisar solo determinados códigos: use `--select` y se le indican los códigos a comprobar:

```
flake8 --select E225,E501 test_linters_inicial.py
```

También puede seleccionar grupos:

```
flake8 --select E,W .
```

3.1.7 7. Ignorar reglas

De forma similar, puede **ignorar** códigos con la opción `--ignore` :

```
flake8 --ignore E501,E303 test_linters_inicial.py
```

3.1.8 8. Ignorar una línea

Es posible ignorar un aviso en una línea concreta:

```
import modulo_no_usado  # noqa
```

> ⚠ **`noqa` no sustituye a corregir el código**
>
> Debe utilizarse solo cuando exista una razón clara para mantener esa línea.

3.1.9 9. Configuración en fichero

Tiene por otra parte la posibilidad de crear un fichero de configuración. Flake8 admite configuración en:

- `.flake8`
- `setup.cfg`
- `tox.ini`

La configuración se coloca en una sección:

```
[flake8]
```

Un posible fichero `tox.ini` en la raiz del proyecto podría ser:

tox.ini

```
[flake8]
ignore = E501,E303
exclude = .git,__pycache__,old,build,dist
max-line-length = 100
max-complexity = 10
```

Después basta ejecutar:

```
flake8 .
```

De forma simplificada, la prioridad de la configuración es:

1. las opciones de línea de comandos tienen prioridad;
2. después se usa la configuración del proyecto;
3. finalmente se aplican los valores por defecto.

3.1.10 10. Ejercicio

Utilizar la herramienta `flake8` con el código de los ejemplos y ejercicios de clase. Revisar y corregir los errores que muestra.

3.1.11 11. Enlaces

- Documentación oficial flake8
- Opciones de flake8: https://flake8.pycqa.org/en/latest/user/options.html

- Códigos de error: https://flake8.pycqa.org/en/latest/user/error-codes.html
- **Descripción con ejemplos** de los códigos de error: https://www.flake8rules.com/

3.1.12 12. Anexo: linter pylint

El objetivo es conocer los linters y su uso básico, y con uno de ellos es suficiente. Pero si lo desea puede probar `pylint` también.

Se instala y se invoca de forma similar a `flake8`

```
(venv) calidad$ pip install pylint
(venv) calidad$ pylint test_linters_inicial.py
************* Module test_linters_inicial
test_linters_inicial.py:55:0: W0301: Unnecessary semicolon (unnecessary-semicolon)
test_linters_inicial.py:63:0: C0325: Unnecessary parens after 'return' keyword (superfluous-parens)
test_linters_inicial.py:66:0: C0325: Unnecessary parens after 'return' keyword (superfluous-parens)
....
-----------------------------------
Your code has been rated at 0.00/10
(venv) calidad$
```

Si desea más información sobre el uso de `pylint` :

- https://pylint.pycqa.org/en/latest/user_guide/run.html
- https://learn.adafruit.com/improve-your-code-with-pylint/pylintrc